

Claude Code session — backend/frontend razor refactor & t3m-swap planning

2026-06-13 17:21 – 03:11 · T3Toolbox · 28 prompts, 284 tool calls

Rendered from the Claude Code transcript at the **readable** level: human + assistant messages verbatim; each tool call collapsed to a one-line summary; internal reasoning and raw tool outputs omitted.

Nick

Hi Claude! We were in the middle of a refactor last night when the power went out and interrupted our work. Over the course of a long day of coding, we started to drift from the design principles of the library, and this refactor was to address one of these drift problems. The particular problem was respecting the frontend/backend split. Frontend code should be a thin OO layer over pure functional backend. The frontend is a convenience for the user, and contains most of the documentation. The backend is where all the math and nontrivial code resides. The one exception to this are the code in `.validate()` methods, because these serve as a contract defining the class. The reason for this split is that an important minority of users will not want to use the OO frontend at all, and we need to support them doing the same functions using the backend only. This refactor has a plan `.md` file: `docs/backend_separation_refactor.md`. I want you to get up to speed with where we were at in the refactor when the power went out, assess anything that went wrong, and report what you find. Then, after I understand where we are at, I will approve you to continue editing.

Claude

I'll get up to speed on the refactor. Let me start by reading the plan document and assessing the current state.

▷ **Read** `docs/backend_separation_refactor.md`

Now let me assess what's actually been changed. Let me look at the git diff and run the tests to see where things stand.

▷ **Bash** — Show diff stat of working tree changes

▷ **Bash** — Show recent commit history

Let me look at the actual diffs for each modified file, starting with the backend additions.

▷ **Bash** — Show diff for backend additions

Now the frontend changes:

▷ **Bash** — Show diff for frontend changes

I now have a clear picture of what was changed. Let me verify the imports the new backend functions depend on, and run the test suite to get ground truth.

▷ **Bash** — Check imports in `orthogonal_representations.py`

▷ **Bash** — Check `ragged_operations` import

Good — `ragged_operations` (alias for `t3_operations`) and `to_numpy` are both available. Let me run the test suite to establish ground truth.

▷ **Bash** — Run frontend test suites

Tests pass (116, green). Now let me determine exactly which frontend rewirings are done vs. pending. Let me inspect the methods that the plan says should be rewired in `manifold.py` and `tucker_tensor_train.py`.

▷ **Bash** — Check `manifold.py` method state

▷ **Bash** — Check `sum_stack` and `save/load` state

None of the `manifold.py` methods reference the new backend functions — confirming only the import line was added there. Let me read the actual bodies of the not-yet-rewired methods to confirm they still hold inline logic (the work that remains).

▷ **Read** `t3toolbox/manifold.py`

The tests are green, but since doctests aren't wired into the runner and the rewiring is incomplete, I need to scrutinize correctness directly. Let me read the not-yet-rewired frontend methods (to confirm what work remains) and the helper signatures the new backend functions depend on.

▷ **Read** `t3toolbox/manifold.py`

▷ **Read** `t3toolbox/manifold.py`

Now I have the full picture of what remains. The `is_gauged` inline (`manifold.py:573-580`) matches the new `gauge_residual` backend exactly, and `normalized/sum_tangents` match their new corewise helpers. Let me now do the plan's required validation step: spot-check the **not-yet-wired** backend functions against the current inline frontend behavior, since those aren't exercised by the passing test suite.

▷ **Bash** — Spot-check not-yet-wired backend functions against inline behavior

All not-yet-wired functions verified correct. Let me also check `corewise_stack_sum` against `sum_tangents`, and confirm the `randn-stream` substitution in the already-wired `randn_variations` is genuinely i.i.d. normal.

▷ **Bash** — Verify `corewise_stack_sum` and `randn` distribution

Both verified. Let me run the remaining test suites (`dispatch` + `contractions`) to complete the regression check.

▷ **Bash** — Run `dispatch` and `contractions` test suites

Let me confirm the exact interruption point and the remaining frontend targets — the `sum_stack_corewise` body and whether the `import ranks` in `manifold.py` is currently unused (the stop marker).

▷ **Read** `t3toolbox/tucker_tensor_train.py`

▷ **Read** `t3toolbox/tucker_tensor_train.py`

I have a complete and verified picture. Here's my assessment.

Where the refactor stopped

The work is **uncommitted** in the working tree (the plan doc `docs/backend_separation_refactor.md` is itself untracked). The plan is a single behavior-preserving commit with two halves: (A) build all the backend functions, (B) rewire each frontend method to delegate. **Half A is 100% done; half B is partially done.** The power cut hit mid-rewire.

The precise stop marker: `manifold.py` has had `import t3toolbox.backend.ranks` as `ranks` added (`manifold.py:18`) but **nothing in the file uses it yet** — you'd just added the import and were about to rewire `manifold_dim/is_gauged` when the power went.

Done and verified (tests green)

All 13 backend functions exist, and the `basis_variations_format.py` rewiring landed:

Plan #	Frontend method	Status
1	<code>T3Basis.is_orthogonal</code> → <code>basis_orthogonality_residual</code>	✓ rewired
3	<code>T3Basis.is_consistent</code> → <code>basis_consistency_residual</code>	✓ rewired
5	<code>T3Basis.has_minimal_ranks</code> → <code>basis_has_minimal_ranks</code>	✓ rewired
6-9	<code>T3Variations.from_vector/zeros/randn/unit</code> → <code>bv_operations.*</code>	✓ rewired

Backend built, frontend NOT yet rewired (the remaining work)

Plan #	Backend fn (present & verified)	Frontend still inline
2	tangent_operations.gauge_residual	T3Tangent.is_gauged (manifold.py:562)
4	ranks.compute_manifold_dim	manifold_dim (manifold.py:27)
10	(via #7/#8)	T3Tangent.zeros/randn (manifold.py:322)
11	corewise.corewise_stack_scale	T3Tangent.normalized (manifold.py:501)
12	corewise.corewise_stack_sum	TuckerTensorTrain.sum_stack_corewise, T3Variations.sum_stack, T3Tangent.sum_tangents
13	common.save_core_families/load_core_families	T3Basis/T3Variations/T3Tangent.save/.load

What went wrong: nothing structural

The refactor was **cleanly interrupted, not left broken**.

- **All tests pass:** 116 (bvf + manifold + ttt) + 34 (dispatch + contractions) = green. The build is additive, so the half-rewired state is consistent.
- I independently verified each of the **six not-yet-wired** backend functions against the current inline implementations, on both an unstacked and a stacked case (these aren't covered by the passing suite yet, so this was the real check):
 - gauge_residual — matches is_gauged's residual exactly.
 - compute_manifold_dim — matches manifold_dim (29).
 - corewise_stack_scale — reproduces normalized().
 - corewise_stack_sum — reproduces sum_tangents for axis $\in \{\text{None}, 0, 1, -1, (0,1)\}$.
 - save_core_families/load_core_families — byte-identical to the existing 'f%d_%d' scheme and round-trips all 6 families.

So the finished half is correct and the pending half is well-defined and low-risk.

Minor flags (not bugs)

1. **randn_variations (item 8, already wired)** now draws from common.randn instead of np.random.randn. Same distribution (verified i.i.d. $N(0,1)$), but the **RNG stream differs**, so any captured random doctest values won't reproduce bit-for-bit. Doctests aren't in CI, so no test impact — just noting it.
2. **Style:** bv_operations.variations_from_vector uses np.prod rather than the house-preferred math.prod — but it's a faithful copy of the inline original, so it's consistent with "behavior-preserving." Optional tidy.
3. **corewise_stack_sum axis convention:** it uses the (axis + n) if axis < 0 form (matching sum_stack_corewise), whereas sum_tangents used axis % k. Identical for valid axes; the backend form is the cleaner one (no silent wrap on out-of-range). Worth a glance when wiring sum_tangents.
4. The stray t3_test_file*.npz artifacts are pre-existing test leftovers (cleanup-backlog items), unrelated to this refactor.

Remaining to finish

Rewire the six pending frontend targets above (manifold.py: 5 methods; bvf: sum_stack + 2 save/load pairs; ttt: sum_stack_corewise; plus T3Tangent.save/load), drop the now-justified import ranks into use, run the full suite, and commit as the single planned commit.

I have **not edited anything** — this is report-only. Let me know when you'd like me to proceed with the remaining rewiring.

Nick

Points 1 and 2 are related to another drift issue. We should not be seeing raw numpy (.np) calls. Everything in the library should support either numpy or jax. The principle is to do dispatch between jax and numpy at the lowest possible level in the code (within reason) using the `tree_contains_jax()`, `get_backend()`, `xnp.` pattern. The dispatch principle is as follows: if any input arguments to a function contain jax, we use jax. Otherwise, we use numpy. Anywhere there is a call to numpy/jax, we need to make a test case in `test_dispatch.py` to ensure that the dispatch is handled correctly. For a reference where this is done correctly, see `backend/probing.py` and `tests/test_dispatch.py`. Please make a note of this but do not fix it yet. We will address this numpy/jax dispatch problem after we finish cleaning up the refactor that was interrupted last night. Point 3: Make a note about this to discuss after the refactor is done. Point 4: Agreed. If you think you understand everything I have written here clearly, you can proceed. If there are any confusions, even small ones, I want you to consult with me first.

Claude

I understand all of it clearly. To confirm my read of each point:

1. **Points 1 & 2 — same drift (raw np.* calls).** The dispatch principle: decide numpy-vs-jax at the lowest reasonable level via `tree_contains_jax()` / `get_backend()` / `xnp`; if any input arg contains jax → jax, else numpy; and every numpy/jax call site needs a corresponding `test_dispatch.py` case. Reference impl: `backend/probing.py` + `tests/test_dispatch.py`. **Note it, don't fix — after the refactor.**
2. **Point 3 — axis-convention difference** (`axis % k` vs `(axis + n) if axis < 0`): note it, discuss after the refactor.
3. **Point 4 — stray .npz artifacts:** agreed, no action.

And critically: the remaining refactor moves stay **behavior-preserving** — I move code as-is, raw `np.*` and all; the dispatch cleanup is a separate later pass.

No confusions. Let me first record the notes, then proceed with the rewiring.

▷ **Edit** `docs/backend_separation_refactor.md`

Notes recorded. Now let me read the exact current bodies of the remaining frontend methods so my edits are precise. Let me read the `bvf save/load/sum_stack` methods and confirm `manifold.py`'s imports.

▷ **Read** `t3toolbox/basis_variations_format.py`

▷ **Read** `t3toolbox/basis_variations_format.py`

Now let me confirm `manifold.py`'s imports (it needs `save_core_families/load_core_families` via `common import *`):

▷ **Read** `t3toolbox/manifold.py`

All imports present. Now let me make the rewiring edits. Starting with `manifold.py` — `manifold_dim`:

▷ **Edit** `t3toolbox/manifold.py`

Now `T3Tangent.is_gauged` (item 2):

▷ **Edit** `t3toolbox/manifold.py`

Now `T3Tangent.normalized` (item 11):

▷ **Edit** `t3toolbox/manifold.py`

Now `T3Tangent.sum_tangents` (item 12):

▷ **Edit** `t3toolbox/manifold.py`

Now `T3Tangent.zeros` and `T3Tangent.randn` (item 10 — delegate to `T3Variations.zeros/.randn`):

▷ **Edit** `t3toolbox/manifold.py`

▷ **Edit** `t3toolbox/manifold.py`

Now `T3Tangent.save/load` (item 13):

▷ **Edit** `t3toolbox/manifold.py`

`manifold.py` done. Now the `bvf` methods — `T3Basis.save/load`:

▷ **Edit** `t3toolbox/basis_variations_format.py`

Now `T3Variations.save/load` and `sum_stack`:

▷ **Edit** `t3toolbox/basis_variations_format.py`

▷ **Edit** `t3toolbox/basis_variations_format.py`

Now the last one — `TuckerTensorTrain.sum_stack_corewise`:

▷ **Edit** `t3toolbox/tucker_tensor_train.py`

All rewiring complete. Let me run the full relevant suite (frontend + dispatch + contractions, as `CLAUDE.md` prescribes after backend-convention changes):

▷ **Bash** — Run full relevant test suite after rewiring

Let me get the test result summary explicitly:

▷ **Bash** — Extract pass/fail summary line

All 150 pass. Since doctests aren't wired into the runner and I changed methods that carry doctests (`manifold_dim`, `sum_stack_corewise`, the new `corewise_*`), let me run doctests on the touched modules to confirm the deterministic captured outputs still hold.

▷ **Bash** — Run doctests on touched modules, surface failures

The doctest "failures" are illustrative fragments referencing names from prior blocks (`x`, `U0`, `base`, stale `t3_corewise_randn`) — pre-existing, per the captured-value convention. But I must confirm my two NEW corewise doctests pass and that I introduced no new failures. Let me check directly by comparing the baseline (stash) against the working tree.

▷ **Bash** — Compare doctest failure counts: working tree vs baseline

Authoritative result from the A/B comparison: **my changes introduced zero new doctest failures** — `corewise` and `manifold` are doctest-clean (0/0), `basis_variations_format` is unchanged (42=42 pre-existing illustrative fragments), and `tucker_tensor_train` actually decreased slightly (397→395). The earlier "21 failed" was an artifact of mixing the `-v` run into the pipe; the clean per-module comparison is definitive. My new `corewise_stack_scale/corewise_stack_sum` and the `manifold_dim` doctest all pass.

Let me get the final diff stat for the report.

▷ **Bash** — Final diff stat for refactor files

The refactor is complete. Here's where things stand.

Refactor finished — all frontend methods now delegate

I recorded the deferred notes you asked for, then completed the interrupted frontend rewiring. Every move was behavior-preserving (code moved as-is, including raw `np.*`).

Newly rewired this session:

#	Method	Now delegates to
2	<code>T3Tangent.is_gauged</code>	<code>tangent_operations.gauge_residual</code>
4	<code>manifold_dim</code>	<code>ranks.compute_manifold_dim</code> (no frontend hop)
10	<code>T3Tangent.zeros / randn</code>	<code>T3Variations.zeros / randn</code>
11	<code>T3Tangent.normalized</code>	<code>corewise.corewise_stack_scale</code>
12	<code>T3Tangent.sum_tangents</code> , <code>T3Variations.sum_stack</code> , <code>TuckerTensorTrain.sum_stack_corewise</code>	<code>corewise.corewise_stack_sum</code>
13	<code>T3Basis / T3Variations / T3Tangent.save/.load</code>	<code>common.save_core_families / load_core_families</code>

Combined with the work already done (items 1, 3, 5, 6–9 in bvf), **all 13 plan items are now wired**, and the frontend holds no math/structural-array logic beyond the agreed exceptions (`.validate()`, `allclose`, `norm`, `bv_to_t3`, `from_canonical`, `size/data_size`).

Verification

- **Full suite green: 150 tests pass** (`test_tucker_tensor_train`, `test_basis_variations_format`, `test_manifold`, `test_dispatch`, `backend/test_contractions`).
- **No new doctest failures** (A/B vs baseline: `corewise` 0/0, `manifold` 0/0, `bvf` 42=42 pre-existing, `ttt` 397→395).
- The not-yet-wired backend functions were independently spot-checked earlier against the inline originals (`gauge residual`, `manifold dim`, `stack-scale`, `stack-sum` across all axis forms, `save/load` byte-identical).

Notes recorded for after the refactor (`docs/backend_separation_refactor.md`, new "Follow-ups" section)

- **(A) numpy/jax dispatch drift** — the `residual/checker` backends (`basis_orthogonality_residual`, `basis_consistency_residual`, `gauge_residual`) and `np.prod` in `bv_operations` use raw `numpy` instead of the `tree_contains_jax/get_backend/xnp` pattern; each call site needs a `test_dispatch.py` case. Reference: `backend/probing.py`. Deferred.
- **(B) sum-axis convention** — `corewise_stack_sum` adopts the no-wrap (`axis + n`) convention (matching `sum_stack_corewise`); the old `sum_tangents` used `axis % k` (silent wrap). To discuss.

One thing to flag

The plan calls for **one commit**, and the work is verified green — but committing/pushing to main is your call, so I've **not committed**. When you're ready I'll stage only the nine refactor files + the plan doc (leaving the stray `.npz/.idea/` artifacts and other unrelated working-tree edits alone) and commit with a descriptive message. Want me to go ahead?

Nick

Go ahead and commit, but afterwards I found some more backend/frontend issues we should discuss before moving on.

Claude

▷ **Bash** — Stage the refactor files and show staged state

Staged exactly the ten refactor files; the stray artifacts are left untracked. Committing:

▷ **Bash** — Commit the refactor

Committed as `068c6cf7`. Per the project workflow (commit per chunk + push to main), pushing:

▷ **Bash** — Push the commit to main

Done — committed as `068c6cf7` and pushed to main (`841ab85a..068c6cf7`). The backend separation refactor is complete: all 13 plan items wired, 150 tests green, no new doctest failures, and the two deferred follow-ups (dispatch drift, sum-axis convention) are recorded in the plan doc.

I'm ready — what are the other backend/frontend issues you found?

Nick

Ok, great. I have looked through manifold.py, and will look through the other files while you are working. Here is what I found in manifold.py. 1) A lot of stacking/unstacking logic is done in the frontend. We need to create backend functions that stack/unstack the T3Tangent basis and variations, and call those from the frontend. 2) project_dense_onto_tangent() checks for jax and converts the array Z unnecessarily. If I'm reading it correctly, this dispatch could be done at a lower level. Please check throughout the validated modules and their dependencies for other problems like this. If we don't actually use jax or numpy functions in a method or function, we don't need to check for jax and do conversions; leave that to subfunctions. If this is clear, you can proceed. If there are any ambiguities, talk through them with me first.

Claude

This is clear, and I have no upfront ambiguities — the investigation itself is unambiguous, and any design questions (e.g. how to structure the backend stacking functions given some already exist in tangent_operations.py) will surface as I read, so I'll raise those in my findings rather than guess now.

Let me start by reading manifold.py's stacking methods and project_dense_onto_tangent, and in parallel grep the validated modules + dependencies for the jax-check anti-pattern.

▷ **Bash** — Find jax-check patterns in frontend validated modules

Let me locate the stacking methods and read the two flagged spots (line 459 and project_dense_onto_tangent at ~920).

▷ **Bash** — Find stacking methods and backend stacking calls in manifold.py

Let me read the stacking methods (828-908) and project_dense_onto_tangent (910-end), plus the line-459 spot:

▷ **Read** t3toolbox/manifold.py

Let me read the method at line 459 and the bvf context at 527:

▷ **Read** t3toolbox/manifold.py

norm() (459) legitimately uses xnp.sqrt/xnp.abs on the result — that's a real array op, so its dispatch is fine, not a violation. Let me check the bvf:517-527 context and audit the backend dependencies for the same anti-pattern (a function that calls get_backend/tree_contains_jax but only passes arrays to subfunctions without doing array ops itself).

▷ **Read** t3toolbox/basis_variations_format.py

random_orthogonal/random_orthogonal_like are legitimate constructors (the documented "pure constructor / factory-from-object infers use_jax" pattern) — not violations.

Now the backend audit. Let me find every dispatch site across the validated backend dependencies and examine whether each function actually uses xnp/np/jnp for array ops, or merely checks-and-forwards.

▷ **Bash** — Find all dispatch sites across backend dependencies

Most of these are the legitimate pattern (the function uses xnp.einsum/etc. right after — contractions.py and probing.py are the reference impls). The anti-pattern is narrow: a function that derives xnp/use_jax but never actually uses it for array ops. Let me write a precise detector — flag any function that calls get_backend but never subsequently references xnp/xmap/xscan.

▷ **Bash** — AST-detect functions that dispatch but never use xnp, or convert-and-forward

No backend function checks-without-using — the dependencies are clean of the anti-pattern. The two conv= hits are the already-noted "raw np" drift (issue A), a different category. Let me run the same detector on the frontend modules to be thorough:

▷ **Bash** — Detect dispatch/convert sites in frontend modules

Now let me verify the project_dense_onto_tangent fix is safe — i.e. that t3svd_dense infers its backend from Z (so removing the conversion can't force the wrong backend) and confirm T3Tangent.project's dispatch:

- ▷ **Bash** — Locate t3svd_dense and project definitions
- ▷ **Read** t3toolbox/tucker_tensor_train.py
- ▷ **Read** t3toolbox/tucker_tensor_train.py

t3svd_dense passes T straight to the dense_t3svd backend with no use_jax — so it infers from the array. Let me confirm the backend truly infers (doesn't default to numpy) and empirically verify the no-conversion path preserves output types across all four numpy/jax combinations before I touch the code:

- ▷ **Bash** — Check dense_t3svd backend dispatch

Confirmed — dense_t3svd dispatches on is_jax_ndarray(T). Now let me empirically verify the no-conversion path preserves the output array type across all four numpy/jax input combinations:

- ▷ **Bash** — Verify no-conversion path preserves output type and values across numpy/jax combos

This surfaces a genuine subtlety I should not silently decide. The empirical result:

dense	basis	OLD output	NEW (no-conv) output	values
np	np	numpy	numpy	match
np	jax	jax	jax	match
jax	np	numpy	jax	match
jax	jax	jax	jax	match

So the conversion is **not** purely cosmetic: in the (jax dense, numpy basis) case the old code *coerces the output down to numpy to match the basis*, whereas removing it follows your stated "any input contains jax → jax" principle. The values are identical in all cases; only the output type differs in that one edge case — and the no-conversion version is the one consistent with the principle (and more jit-robust: to_numpy on a tracer would actually fail). This is exactly the kind of call I should bring to you rather than decide.

Before I write up the full findings, let me read the four backend stacking functions so I can frame issue #1 precisely.

- ▷ **Read** t3toolbox/backend/tangent_operations.py

I now have everything. Let me read apply_func_to_leaf_subtrees briefly to confirm the frontend's leaf-structure args are the movable part, then I'll write up findings.

- ▷ **Bash** — Locate the stacking tree-machinery functions
- ▷ **Read** t3toolbox/backend/stacking.py

I've completed the audit and analysis. Here are my findings on both issues. I have **not edited anything** — each has a decision for you.

Issue #2 — the "checks jax but does no array math" anti-pattern

I swept all four validated frontend modules and twelve backend dependency modules (tangent_operations, bv_conversions, bv_operations, t3_operations, t3_linalg, t3_orthogonalization, linalg, probing, ranks, contractions, orthogonal_representations, stacking) with an AST scan that flags any function calling get_backend/tree_contains_jax/to_jax/to_numpy that never goes on to use xnp/xmap/xscan.

Result: project_dense_onto_tangent is the only genuine instance. Everything else that dispatches is legitimate:

- T3Tangent.norm — actually uses xnp.sqrt/xnp.abs on the inner product. Real array op; dispatch needed.
- T3Tangent.zeros/randn, T3Basis.random_orthogonal*, the three loads, zeros_like/randn_like — these infer use_jax and **thread it into a pure constructor** (or .to_jax() a freshly-loaded npz). That's the documented "factory infers from the existing object, pure constructor takes the flag" pattern, not the anti-pattern.

- **Backend dependencies: zero check-but-forward cases.** (The two `to_numpy/to_jax` hits there — `basis_consistency_residual`, `unit_variations` — are the *separate* “raw np.*” drift already noted as follow-up (A) from the last commit; those functions *do* compute with numpy directly, a different category.)

The decision for `project_dense_onto_tangent`. Your read (“converts Z unnecessarily”) is right in 3 of 4 cases — but the conversion isn’t a pure no-op. Current line:

```
Z = to_jax(dense_tensor) if tree_contains_jax(basis.data) else to_numpy(dense_tensor)
```

This coerces Z to **match the basis**. In the (jax dense, numpy basis) case it pulls a jax array *down* to numpy, so the output is numpy. Pushing dispatch down instead (drop the line, pass `dense_tensor` straight to `t3svd_dense`, which infers from its input; project then unifies) makes the output **jax** there — which is what your “any input contains jax → jax” principle dictates, and is more jit-robust (`to_numpy` on a tracer would raise). Values are identical in all four combos.

My recommendation: remove the conversion (principle-aligned). The only behavior change is that one mixed edge case correctly becomes jax instead of numpy. Confirm and I’ll apply it + add a `test_dispatch.py` case covering `project_dense_onto_tangent` for both input types.

Issue #1 — stacking/unstacking logic in the frontend

Here’s the wrinkle: the **array-axis stacking is already in the backend** — `tangent_operations.{stack,unstack}_{tangent,base}_stack` operate on trees of raw data tuples and own the V/G split recovery and the axis math. The four `T3Tangent` methods already call them. What actually remains in the frontend is:

1. **Object ↔ data wrap/unwrap** (`apply_func_to_leaf_subtrees(tree, lambda t: t.variations.data, ...)` and the reverse) — *inherently* frontend, since it constructs `T3Tangent/T3Basis/T3Variations`.
2. **Hardcoded leaf-structure layout specs** — `((None,)*self.d, (None,)*self.d), ((None,)*self.d,)*4`. This is backend data-layout knowledge (how many families, how many cores each) **leaking into the frontend**. This is the part I’d most want to move.
3. `tree_zip` orchestration (in `unstack_basis`) to pair the basis/variation object-trees.
4. **Structural validation** — the same-`T3Basis`-identity check (`stack_tangents`) and structure/tangent-stack-match check (`stack_basis`). Per the house philosophy these are contract-defining guards that *stay* in the frontend.

So before I design this, I need to know which of these you mean by “a lot of stacking logic in the frontend.” My reading is that the real target is #2 (and the #1/#3 orchestration around it) — the wrap/unwrap and validation legitimately stay. Two ways to attack it:

- **(a)** Backend gains thin “tree” entry points that take/return *trees of data* and internally own the leaf-structure knowledge, so each frontend method becomes `unwrap → backend_tree_op → wrap` with no hardcoded `(None,)*self.d` and no `tree_zip` in the frontend.
- **(b)** Leave the four methods as-is but have the backend *expose* the leaf-structure descriptors (e.g. `T3Variations/T3Basis` data layout), so the frontend stops hand-rolling `(None,)*self.d`.

Which did you have in mind — and do you agree the object wrap/unwrap and the validation guards should remain in the frontend, or do you want those reconsidered too? Once I know the target I’ll propose the concrete backend signatures.

Nick

Agree with the recommendation for `project_dense_onto_tangent`. Make sure to add appropriate dispatch tests. Any code which does follow the dispatch rule should be considered a bug and flagged. For the stacking/unstacking, I would prefer to talk through the principle with you until you understand, rather than giving orders on specific cases. The principle is this: we want a user who is not using the frontend to be able to perform the stacking/unstacking without much deliberation or work. If the function is trivial, we don’t need to put it in the backend. If the function requires some nontrivial level of logical thought, we should add it to the backend. The deciding question is: would an average human

coder find it easier to search through the backend, find the appropriate backend function, and use it, or would they find it easier to simply rewrite what the function does on their own.

Claude

Understood on both. And yes — I'll read your "does **not** follow the dispatch rule → bug, flag it" as the operative severity (the residual/checker functions from follow-up (A) are now on the bug list, not the style list).

Let me first land the approved `project_dense_onto_tangent` fix with dispatch tests, then talk through the stacking principle. Let me look at the `test_dispatch.py` pattern and how `project/manifold` ops are covered there.

▷ **Bash** — Survey `test_dispatch.py` structure

`project_dense_onto_tangent` calls `t3svd_dense` with no truncation — ranks are chosen from the data's SVD, so shapes are dynamic and it can't be jitted. It belongs in the eager "jax-in → jax-out" bucket. Let me read that section and the setup:

▷ **Read** `tests/test_dispatch.py`

Let me check `STRUCT` and the imports at the top of the test file:

▷ **Read** `tests/test_dispatch.py`

Now I'll implement the approved fix and add dispatch tests. First, the `project_dense_onto_tangent` fix:

▷ **Edit** `t3toolbox/manifold.py`

Now the dispatch tests — adding to the dynamic-shape bucket (since `t3svd_dense` picks ranks from the data):

▷ **Edit** `tests/test_dispatch.py`

Let me run the dispatch and manifold suites to verify:

▷ **Bash** — Run dispatch and manifold tests after the fix

All 42 tests pass. The fix is in and the dispatch tests (including the mixed jax dense + numpy basis → jax regression guard) pass. Let me commit this self-contained, approved chunk before we move into the stacking discussion.

▷ **Bash** — Check working tree for the dispatch fix files

▷ **Bash** — Commit and push the dispatch fix

Committed and pushed (1fc7bf0f). Now — the stacking principle. Let me reflect it back and apply it, so you can check whether I've actually got it.

The principle, as I understand it

The backend exists so that **a backend-only user doesn't have to re-derive anything nontrivial**. So the test for "does this belong in the backend?" is not "does it touch arrays?" and not "is it OO-flavored?" — it's a **cost comparison from the perspective of someone who isn't using the frontend**:

Faced with needing this operation, would an average coder rather (a) go find a backend function and call it, or (b) just rewrite it themselves?

- If (b) — the logic is trivial enough that searching/finding/trusting a backend function is *more* work than just writing it. Leave it; don't bloat the backend.
- If (a) — the logic takes real thought to get right (and is easy to get *wrong*), so its absence forces the backend user to reinvent something error-prone. That must live in the backend.

The decision is about **the difficulty of the logic**, judged by the rewrite-vs-reuse tradeoff — not about where the code happens to sit now.

Applying it to the four stacking methods

Each method decomposes into three kinds of work. Let me classify each by the principle:

1. The actual stack/unstack of the cores — recovering the K/C (tangent/base) split from shapes (`_tangent_stack_split`), then peeling/stacking the *right* axis ranges under the base-inner K + C convention. This is exactly the “most error-prone part of the library.” A backend user would *badly* want to find this rather than rewrite it — getting the axis ranges or the inner/outer ordering wrong is a silent bug. → **Backend. And it already is** (`tangent_operations.{stack,unstack}_{tangent,base}_stack + _tangent_stack_split`). ✓

2. Wrapping the resulting data-trees back into T3Tangent/T3Basis/T3Variations (the `apply_func_to_leaf_subtrees(..., lambda d: T3Tangent(...))` calls). A backend user *never does this* — they have raw data tuples, not objects. There’s nothing for them to rewrite or reuse. → **Frontend, legitimately** (and it *can’t* move — the backend can’t depend on the frontend classes). ✓

3. The structural guards — the same-T3Basis-object-identity check in `stack_tangents`, the structure/K-match check in `stack_basis`. These are about *object identity*, which only exists in the frontend, and they’re contract-defining. → **Frontend** (consistent with the `validate-stays-in-frontend` exception). ✓

My honest conclusion (and where I’d push back)

Run through the principle, **the genuinely hard logic is already in the backend and is findable/reusable** — a backend user round-trips cleanly with `unstack_base_stack ↔ stack_base_stack` (and the tangent-stack pair), all in terms of (`basis_tree`, `variations_tree`) data. The code left in the frontend is object-wrapping and identity-validation, which a backend user neither needs nor would rewrite.

So under your own test, the split looks *mostly* right already. The one spot I’d actually scrutinize is **unstack_basis**: the backend hands back *two parallel trees* (`basis_tree`, `variations_tree`), and the frontend then does a `tree_zip` + double-wrap dance. A backend user wanting per-base-point (`basis`, `variations`) pairs has to know to call `tree_zip` themselves — minor, but it’s the only place the backend stops slightly short of “call one function, done.”

So my question back, to confirm I’ve understood: when you said “a lot of stacking logic is in the frontend,” did you mean —

- (i) the **hard axis/split logic** (which I’m finding is *already* in the backend — in which case maybe the real gap is discoverability, not missing code), or
- (ii) the **tree-orchestration dance** (`apply_func_to_leaf_subtrees` + hand-built `leaf_structure` + `tree_zip`) — which is nontrivial-looking but is inseparable from producing frontend objects, so the most it can shrink is by the backend returning friendlier shapes (e.g. `unstack_base_stack` yielding a paired tree)?

If it’s (ii) and you’d still call that dance “nontrivial logic a backend user would rewrite,” then I think I’m *missing* something about who that logic serves — and I’d want you to point at it, because by my read a backend user bypasses it entirely.

Nick

Ok, you understand my intentions and I agree with your pushback. Logic which is inseparable from the frontend should stay in the frontend. As you point out, this is what most of the stacking method logic is doing. However, I would like to have an `unstack_basis` backend which does the `tree_zip` (the user may not know about `tree_zip`). More broadly, this backend/frontend split concept and razor (what is easiest for a pure backend user) applies to the whole project.

Claude

Understood — and I agree this generalizes. Let me check the blast radius of the two base-stack functions before I decide how far to take it, then record the razor as a project-wide principle.

▷ **Bash** — Find all callers of the base/tangent stack backend functions

Only the four frontend methods call these — no tests touch them directly, so the blast radius is just `manifold.py`. Let me look at `tree_zip` and the tree utilities in `stacking.py` to see how cleanly the pairing (and a potential inverse) can be done in the backend:

▷ **Read** `t3toolbox/backend/stacking.py`

`tree_zip` recurses until a non-Sequence leaf — that's exactly why the current frontend wraps data into `T3Basis/T3Variations` *objects* first (objects are opaque leaf-markers); zipping the raw **data** trees would over-recurse into the core tuples. So a correct backend zip needs leaf/depth awareness — which confirms this is genuinely nontrivial (a backend user would hit the over-recurse bug). Let me check how `trees_have_same_structure` decides a leaf, since both the backend zip and the frontend wrap depend on it being unambiguous:

▷ **Bash** — Locate tree structure helpers

▷ **Read** `t3toolbox/backend/stacking.py`

Now I understand the full shape of it. The key finding: `tree_zip` recurses to non-Sequence leaves, but data-tuple leaves *are* sequences — so a backend user pairing the two trees would hit an over-recurse bug. That's genuinely nontrivial logic (it needs the base-stack depth), which is exactly why it belongs in the backend per your razor.

I'll make `unstack_base_stack/stack_base_stack` **symmetric** around a paired tree ((`basis_data`, `variations_data`) leaves) — that keeps them true inverses, so a backend user round-trips with neither `tree_zip` nor a hand-rolled unzip. Both frontend methods get simpler too. Let me implement the backend helpers and rewrite the two functions:

▷ **Edit** `t3toolbox/backend/tangent_operations.py`

▷ **Edit** `t3toolbox/backend/tangent_operations.py`

Now the frontend `unstack_basis` and `stack_basis`:

▷ **Edit** `t3toolbox/manifold.py`

▷ **Edit** `t3toolbox/manifold.py`

Now let me verify — run the manifold suite (which exercises stack/unstack round-trips) plus dispatch:

▷ **Bash** — Run manifold and dispatch tests after stacking refactor

All 42 pass. Let me do an explicit round-trip check across the tricky cases — `G=()`, `G=(m,)`, multi-axis `G=(m1,m2)`, and with a tangent stack `K` present — to be sure the depth-aware pairing and the -3 depth derivation are correct in all of them:

▷ **Bash** — Explicit stack/unstack round-trip across `C`, `K` combinations

All round-trips correct, including multi-axis `C=(2,3)` and with `K` present. Now let me record the razor as a project-wide principle in `CLAUDE.md` (the source of truth), in the Architecture section:

▷ **Edit** `CLAUDE.md`

Let me run the full relevant suite (the stacking signature change is a backend-convention change, so per the project guidance I run all the consumers' suites, not just the touched one):

▷ **Bash** — Run full relevant suite after stacking change

All 150 pass. Let me confirm no doctest regressions on the two touched modules (`A/B` vs the committed baseline):

▷ **Bash** — `A/B` doctest check on touched modules

No doctest regressions. Everything's green. I've **not committed** — because I extended past your literal ask (you requested the unstack side; I also made the stack side symmetric), so I want you to sign off on that before it lands.

What I did

Backend (`tangent_operations.py`) — paired-tree API. `unstack_base_stack` now returns a C-shaped tree whose leaves are (`basis_data`, `variations_data`) pairs (the `tree_zip` is folded in), and `stack_`

`base_stack` takes that same paired tree back. They're now true inverses. The pairing is a small depth-aware helper (`_pair_base_leaves/_unpair_base_leaves`) rather than `tree_zip` — precisely because `tree_zip` would over-recurse into the data-tuple leaves, which is the nontrivial bit a backend user shouldn't have to rediscover.

Frontend (`manifold.py`) — both methods shrink:

- `unstack_basis`: was *unwrap*→*backend*→*wrap basis*→*wrap variations*→*tree_zip*→*wrap pairs* (5 steps); now *backend*→*one wrap*.
- `stack_basis`: was two extraction passes → backend; now one extraction pass → backend.

CLAUDE.md: recorded the razor as a project-wide principle (pure-backend-user perspective; find-and-reuse vs rewrite; the inseparable-from-frontend exception; the "don't leave them one fiddly step short" corollary).

Two things for your review

1. **The stack-side extension is my call, not your ask.** My reasoning: if `unstack_base_stack` returns a paired tree but `stack_base_stack` still took two separate trees, a backend user round-tripping would have to *unzip* the pairs by hand — the exact "might not know the helper" friction you flagged, just mirrored onto the stack side. Making them symmetric means `stack_base_stack(unstack_base_stack(b, v)) == (b, v)` with no tree gymnastics either direction. Low risk: the only callers are the two frontend methods. If you'd rather keep stack taking two trees, say so and I'll dial it back.
2. **One bit of fragility I want to flag honestly:** `stack_base_stack` recovers the base-stack depth as `tree_depth(paired_tree) - 3` (a pair-leaf is 3 sequence levels deep). It mirrors the existing `- 2` magic in the old code, and it's covered by the round-trip tests, but it's the kind of structural constant that a future core-layout change could silently break. Acceptable to me given the test guard, but noting it.

Verification

- Full suite green (150) + the dispatch suite.
- Explicit round-trips correct for $C \in \{(2,), (2, 3)\} \times K \in \{(), (4,)\}$ and the tangent-stack side — including multi-axis base stacks and K present; leaves come back as single-base-point tangents with the right K , shared-base identity preserved.
- No new doctest failures on either touched module.

Want me to commit (`tangent_operations.py` + `manifold.py` + **CLAUDE.md**), and is the symmetric stack-side change what you want?

Nick

Keep the symmetric change and commit it

Claude

- ▷ **Bash** — Confirm working-tree changes before staging
- ▷ **Bash** — Commit and push the symmetric stacking change

Committed and pushed (1dabf921).

To recap this session:

- **1fc7bf0f** — fixed the `project_dense_onto_tangent` dispatch bug (pushed the jax/numpy decision down to the subfunctions) + dispatch tests including the `jax-dense/numpy-basis` → `jax` regression guard.
- **1dabf921** — moved the base-stack tree-pairing into the backend as a symmetric paired-tree API (`unstack_base_stack/stack_base_stack` are now true inverses), simplified the two frontend methods, and recorded the backend/frontend razor in **CLAUDE.md**.

Still on the table from earlier (deferred, not lost):

- The **dispatch-rule bug list** (now treated as bugs, not style): the raw-np.* residual/checker backends — `basis_orthogonality_residual`, `basis_consistency_residual`, `gauge_residual` — plus the `np.prod` static-product spots in `bv_operations`. These are recorded in `docs/backend_separation_refactor.md` under follow-up (A); ready to sweep when you want.

You mentioned you'd be reviewing the other files in parallel — happy to take whatever you found next, or start on that dispatch bug list.

Nick

Let's try to apply the backend / frontend razor to all of the methods in all of the validated modules. Some that I flagged are: saving/loading mechanics, `random_orthogonal()`, and `reverse()`. Please read through the validated modules and assess methods and functions on a case by case basis, to see if anything ought to be moved. Make a list of suggested moves, along with your reasoning, and present it to me. Then we will talk it over, before proceeding.

Claude

This is a read-and-assess task across the three OO frontend modules — I'll produce a curated list of suggested moves with reasoning, and **not edit anything**. (`corewise.py` is already a pure-functional module — it has no OO frontend, so the razor doesn't apply to it; I'll note that and focus on `tucker_tensor_train.py`, `basis_variations_format.py`, and `manifold.py`.)

To cover three large modules thoroughly and consistently, I'll fan out one reader per module to inventory every method and classify its body (pure delegation vs. trivial inline vs. nontrivial inline logic), then I'll apply the razor myself to the candidates — verifying the strongest ones by reading their actual bodies before they go on the list.

- ▷ **Agent** (general-purpose) — Assess `tucker_tensor_train.py` methods
- ▷ **Agent** (general-purpose) — Assess `basis_variations_format.py` methods
- ▷ **Agent** (general-purpose) — Assess `manifold.py` methods

Three thorough audits in. Before I curate, let me verify the strongest candidates I haven't personally read — `TuckerTensorTrain.save/load` and `segment/concatenate` — and re-confirm `T3Basis.reverse`:

- ▷ **Read** `t3toolbox/tucker_tensor_train.py`
- ▷ **Read** `t3toolbox/tucker_tensor_train.py`
- ▷ **Bash** — Find module-level convenience functions in TTT

No module-level functions are defined in `tucker_tensor_train.py` — the `t3.t3_save/t3_corewise_randn` references in those docstrings are stale (a pre-refactor module API), a doctest-hygiene issue separate from this audit. Let me confirm `segment/concatenate` and `T3Basis.reverse`:

- ▷ **Read** `t3toolbox/tucker_tensor_train.py`
- ▷ **Read** `t3toolbox/tucker_tensor_train.py`

I've verified the key bodies myself. Here's my curated assessment. I've applied the razor independently to the agents' findings (and I differ from them on a couple). **No edits made** — this is for discussion.

First, scope notes: `corewise.py` is excluded (it's already a pure-functional module, no OO frontend). And of your three flagged items, the `save/load` and `reverse` story is more specific than it first looks: the `T3Basis/T3Variations/T3Tangent save/load` **already** delegate to `save_core_families` (from the refactor); only `TuckerTensorTrain`'s does not. And only `T3Basis.reverse` (of the four reverses) has nontrivial logic.

Strong — clear moves

1. `TuckerTensorTrain.save / load` → `reuse common.save_core_families/load_core_families`. This is the standout. It hand-rolls a *different* `npz` scheme (`tucker_cores_{i}/tt_cores_{i}`) from the `f{fam}_`

{core} scheme every other class now uses, infers the family split as `len(files)//2` with a bare assert, and swallows `RuntimeError` then falls through to use an undefined `d` (a real bug on the error path). `self.data` is exactly two families, so `save_core_families(file, self.data) / load_core_families` drops in cleanly — same pattern as `T3Variations`. Pure serialization of `.data` is backend work, the backend already exists, and a bypass user would obviously rather call the shared helper.

- **Caveat (your call):** unifying changes TTT’s on-disk key names, so old TTT `.npz` files won’t load. Alternative is to move the *existing* scheme into a backend fn (preserves files, but keeps two schemes around). Given the repo is pre-release (“DO NOT USE”), I’d unify — but flagging it.

2. `T3Basis.reverse` → new backend fn `reverse_basis(basis_data)`. The per-family `reverse_tt` calls are delegation, but the **left→right role-swap** (old-right → new-left slot, old-left → new-right) is genuine structural+math knowledge: it depends on the (up, down, left, right) ordering and on the fact that reversing a left-orthogonal chain yields a right-orthogonal one. No backend fn does this (`reverse_tt` is one family). A bypass user would very plausibly wire the swap backwards. Natural home: `bv_operations.py`. (`T3Variations.reverse` and `TuckerTensorTrain.reverse` are genuinely trivial — no swap — and stay; optionally add a parallel `reverse_variations` only for symmetry, not needed by the razor.)

Medium — defensible, want your read

3. `T3Tangent.norm` → new `corewise.corewise_stack_norm(X, n_stack)`. `norm` is `sqrt(abs(inner(self,self)))` computed inline with `xnp`. The existing family has `corewise_stack_dot/_scale/_sum` but **no `_stack_norm`** — and the one that exists, `corewise_norm`, *collapses the stack* (a trap: a bypass user reaching for it gets the wrong shape). Adding `corewise_stack_norm` rounds out the family and gives the vectorized norm a findable home. The logic is light (it’s `sqrt` of `corewise_stack_dot(X,X,n)`), so this is more about family-completeness + trap-avoidance than complexity.

4. `TuckerTensorTrain.concatenate` (and `segment`) → `t3_concatenate / t3_segment` in `t3_operations.py`. `concatenate` carries a real domain invariant — the seam TT-rank-compatibility check (`x.tt_ranks[-1] == y.tt_ranks[0]`) — plus a recursive fold, all on raw core tuples; that’s the compelling one. `segment` is its inverse but weaker (mostly `None`/negative-index normalization + a bounds check), so I’d rate it lower than the agent did — borderline. They’re See-Also-linked inverses, so I’d move them together or not at all.

5. `T3Tangent.retract` → `tangent_operations.retract(basis_data, variations_data)`. Each sub-step delegates (to `t3(include_shift=True)`, then `t3svd`), but the load-bearing choice — *truncate back to the base point’s own ranks* (`basis.up_ranks/left_ranks`), i.e. Algorithm 10 — is frontend-only. Retraction is a fundamental named manifold operation; a bypass user doing optimization would want to *find* it, not reconstruct the “which ranks” recipe. The argument here is discoverability of a named operation more than logic-complexity.

Recommend leaving (incl. one you flagged)

- **`T3Basis.random_orthogonal` (you flagged it):** honestly, it’s a 2-line composition — `randn(...)` then `t3_orthogonal_representations(...)[0]`, both already backend/factory calls. By the razor that’s trivial; a bypass user composes the same two calls. The only argument *for* moving is discoverability of the recipe “random orthogonal = `randn` → orthogonalize” as a named `random_orthogonal_representation` backend fn. I lean leave-it, but since you flagged it, tell me if you want the named convenience.
 - **`T3Tangent.tangent_stack_shape`:** reimplements the $K = \text{full} - C$ split that `tangent_operations._tangent_stack_split` already does privately. One trivial line; at most, make that helper public and reuse it.
 - **`TuckerTensorTrain.inner` (ndarray branch):** a small stack-aware contraction `inds + (to_dense()*other).sum(...)`. Minor; low priority.
 - The `__add__/_mul__`/inner operator dispatch (isinstance + structural guards) stays — that’s frontend operator semantics; the math already delegates to `t3_add/t3_mult/t3_scale`.
-

Side notes (not razor moves, surfaced during the read)

- The save/load swallowed-`RuntimeError`-then-`undefined`-d path is a genuine bug — the move in #1 fixes it for free.
- Stale doctests: `TuckerTensorTrain.save/load` docstrings call module-level `t3.t3_save/t3_load/t3_corewise_ran` that don't exist in this module; apply's doctest still passes `use_jax=True`. Doctest hygiene, separate cleanup.
- The dispatch-rule bug list (residual/checker functions using `raw np.*`) is still tracked in the refactor doc — unrelated to these moves.

My ranking if we proceed: **#1 and #2 are clear; #3 is easy and tidies the corewise family; #4/#5 are judgment calls** I'd like your read on. Which of these do you want to take, and on #1 do you want to unify the save scheme (breaking old TTT files) or preserve it?

Nick

I agree with 1,2,3,4,5. I agree with your recommendations on leaving, including your pushback on `random_orthogonal`. For 1, I want to unify the save scheme. Don't worry about breaking old files. If there are no ambiguities, feel free to proceed

Claude

Approved — I'll proceed on all five, unifying the save scheme. Let me first confirm a few implementation details (not user-facing ambiguities — code facts I need to get the moves right): the `.npz`-handling in the family helpers vs. TTT's current behavior, where `reverse_basis` should import `reverse_tt`, the retract building blocks, and `corewise_stack_dot`'s signature.

▷ **Read** `t3toolbox/backend/common.py`

▷ **Bash** — Gather implementation details for moves 1,2,5

`load_core_families` does a bare `np.load(file)` — no `.npz` appending, unlike TTT's current `load`. So I need to check how the sibling classes' save/load tests pass filenames, to keep the unified TTT behavior consistent. Let me check the tests, `corewise_stack_dot`'s signature, `tangent_to_t3`'s signature, and rule out circular imports for the two new backend imports:

▷ **Bash** — Check save/load tests, signatures, and circular imports

All resolved cleanly: every save/load test passes explicit `.npz` filenames, so dropping TTT's bespoke `.npz`-appending (to match the siblings) breaks nothing; no circular imports for the two new backend imports; signatures confirmed. Let me read the TTT save/load test and confirm the `t3_svd.t3svd` call shape before I implement:

▷ **Read** `tests/test_tucker_tensor_train.py`

The TTT test just round-trips and compares cores — unifying is safe. Let me confirm the `t3svd` backend call shape used by retract:

▷ **Read** `t3toolbox/tucker_tensor_train.py`

The TTT test only round-trips, and the `t3svd` backend returns `(data, ss, ss)`. All implementation facts confirmed; no user-facing ambiguities. Let me read the exact current bodies I'll edit (`T3Basis.reverse`, TTT load tail) so my edits are precise, then implement all five moves.

▷ **Read** `t3toolbox/basis_variations_format.py`

▷ **Read** `t3toolbox/tucker_tensor_train.py`

I have all the exact bodies. Let me nail the insertion points and import aliases, then implement all five moves.

▷ **Bash** — Locate import aliases and insertion points

▷ **Read** `t3toolbox/corewise.py`

▷ **Read** `t3toolbox/backend/t3_operations.py`

▷ **Bash** — Read end of `tangent_operations.py` for retract insertion

All insertion points confirmed. Implementing the five moves now. **Move 2 — reverse_basis in bv_operations.py:**

- ▷ **Edit** t3toolbox/backend/bv_operations.py
- ▷ **Read** t3toolbox/backend/bv_operations.py
- ▷ **Edit** t3toolbox/backend/bv_operations.py
- ▷ **Edit** t3toolbox/backend/bv_operations.py

Move 3 — corewise_stack_norm in corewise.py:

- ▷ **Read** t3toolbox/corewise.py
- ▷ **Edit** t3toolbox/corewise.py
- ▷ **Edit** t3toolbox/corewise.py

Move 4 — t3_segment/t3_concatenate in t3_operations.py:

- ▷ **Read** t3toolbox/backend/t3_operations.py
- ▷ **Edit** t3toolbox/backend/t3_operations.py
- ▷ **Edit** t3toolbox/backend/t3_operations.py

Move 5 — retract in tangent_operations.py:

- ▷ **Read** t3toolbox/backend/tangent_operations.py
- ▷ **Edit** t3toolbox/backend/tangent_operations.py
- ▷ **Edit** t3toolbox/backend/tangent_operations.py

Backend additions done. Now the frontend rewiring. **tucker_tensor_train.py — save, load, segment, concatenate:**

- ▷ **Edit** t3toolbox/tucker_tensor_train.py
- ▷ **Edit** t3toolbox/tucker_tensor_train.py
- ▷ **Edit** t3toolbox/tucker_tensor_train.py
- ▷ **Edit** t3toolbox/tucker_tensor_train.py

Now basis_variations_format.py — T3Basis.reverse:

- ▷ **Edit** t3toolbox/basis_variations_format.py

Now manifold.py — T3Tangent.norm and retract:

- ▷ **Edit** t3toolbox/manifold.py
- ▷ **Edit** t3toolbox/manifold.py

All five moves implemented. Let me first verify the package imports cleanly (the two new backend imports could in principle create a cycle), then run the full suite.

- ▷ **Bash** — Check for circular imports

No import cycles. Now the full relevant suite:

- ▷ **Bash** — Run full relevant suite after the five moves

All 150 pass, including the jit-bucket tests for norm and retract (proving the new backend paths have no numpy leak). Let me verify the new corewise_stack_norm doctest value, confirm the save scheme actually unified, and run an A/B doctest check on the touched modules:

- ▷ **Bash** — Verify doctest value, unified save scheme, and A/B doctest counts

The doctest value and unified save scheme are correct, but **I introduced 4 new doctest failures in tucker_tensor_train.py (197 → 201)**. I need to find and fix those. Let me pinpoint them by diffing the failed examples against baseline:

- ▷ **Bash** — Diff failing doctest examples vs baseline

The regression is clear: the load method's docstring doctest uses a **bare filename** ('t3_file') and relied on the .npz-appending I just removed for unification — so it now fails (and save's doctest still uses the stale t3.t3_save/t3_load module funcs). Since I changed save/load's behavior, the doctests must be updated to match. Let me read both docstrings fully:

▷ **Read** t3toolbox/tucker_tensor_train.py

Found it: the load docstring uses a bare 't3_file' and relied on the .npz-appending I removed, and save's docstring still calls the stale t3.t3_save/t3_corewise_randn module funcs. Since I changed save/load's behavior, the doctests must match the new API. Fixing both (real method API + explicit .npz):

▷ **Edit** t3toolbox/tucker_tensor_train.py

▷ **Edit** t3toolbox/tucker_tensor_train.py

Let me re-verify the doctest count is now back to baseline-or-better (and clean up the t3_file.npz the save doctest writes):

▷ **Bash** — Re-check doctest count vs baseline and clean stray file

Working tree (190) is **below** baseline (196) — so no new doctest failures, and I actually fixed 6 pre-existing stale ones in the save doctest. Stray file cleaned. The doctest edits were docstring-only, but let me run the full suite once more to confirm everything's green after all edits:

▷ **Bash** — Final full suite run + ensure no stray files

All 150 tests pass (the lingering t3_test_file*.npz are pre-existing strays from the original working tree, not mine). All five moves are done and verified. Let me stage exactly the touched files and commit.

▷ **Bash** — Confirm exactly the touched files changed

▷ **Bash** — Commit and push the five razor moves

Committed and pushed (15d8910a).

Summary

All five razor-driven moves are landed and verified. Each frontend method is now a thin delegate; the nontrivial logic lives in the backend where a pure-backend user can find it:

Move	Backend home (new)	What moved
1	common.save_core_families/load_core_families	TTT save/load — unified onto the shared f{fam}_{core} scheme (dropped the bespoke scheme + its assert/swallowed-error bug)
2	bv_operations.reverse_basis	the left↔right role-swap on basis reversal
3	corewise.corewise_stack_norm	the stack-preserving HS norm (rounds out the stack_* family)
4	t3_operations.t3_segment/t3_concatenate	index-normalization + the seam TT-rank invariant
5	tangent_operations.retract	the retraction recipe (shift-embed → T3-SVD to the base's ranks)

Verification: 150 tests green; no import cycles; corewise_stack_norm doctest value correct; save scheme confirmed unified (f0_*/f1_* keys, round-trips, jax-load works); **no new doctest failures** (190 vs 196 baseline — I also fixed 6 pre-existing stale ones in the save doctest by updating it to the real method API). Left as-is per the razor: random_orthogonal, operator dispatch, validation/identity guards.

Open items still tracked (not touched here):

- The dispatch-rule **bug list** — residual/checker backends (basis_orthogonality_residual, basis_consistency_residual, gauge_residual) using raw np.*, plus np.prod static-product spots — recorded in docs/backend_separation_refactor.md.
- Broader **doctest hygiene** (190 remaining failures in tucker_tensor_train.py are mostly stale module-level t3.t3_corewise_randn-style references; the apply doctest's use_jax=True).

Want me to pick up the dispatch bug list next, or are there more files/methods you flagged in your review?

Nick

go ahead with the dispatch bug list

Claude

On it. Let me first do a comprehensive sweep for raw `np.*` array-computation calls across the validated modules and their verified backend dependencies, so I catch everything — not just the three recorded residual functions. Then I'll triage genuine dispatch bugs from legitimate uses (I/O, static products, coercion) and fix + test.

▸ **Bash** — Find raw `np.*` calls in validated modules and deps

Let me triage the non-obvious hits — the `t3_linalg np.ones`, and check which files already import `math` (for the `np.prod` → `math.prod` style fixes):

▸ **Bash** — Inspect `t3_linalg np.ones` and check `math` imports

`t3_linalg:188` works under `jit` (numpy constants fold) but still violates the rule — I'll fix it to `xnp.ones`. I omitted `tucker_tensor_train.py` from the sweep; let me check it for raw `np.*` too (the agent noted `.size` uses `np.prod`):

▸ **Bash** — Check `tucker_tensor_train.py` for raw `np` and inner dispatch coverage

Triage complete. The genuine dispatch bugs: the 3 residual functions (raw `np.*` array computation — must dispatch via `xnp` and be `jit`-able) and `t3_linalg:188 np.ones`. The `np.prod` spots (`.size` properties + `variations_from_vector`) are static shape products → `math.prod` per house style. `np.minimum` in ranks (integer rank arithmetic), `np.broadcast_shapes` (tuple shape arithmetic), `unit_variations/save/load` (constructors/I/O) are legitimately not dispatch bugs.

Let me read the exact bodies I'll rewrite:

▸ **Read** `t3toolbox/backend/orthogonal_representations.py`

Now rewriting the two `orthogonal_representations.py` residuals to dispatch via `xnp` and be `jit`-able (collect scalar devs → `xnp.stack` → `xnp.max`, no `float()/Python max()`):

▸ **Edit** `t3toolbox/backend/orthogonal_representations.py`

▸ **Edit** `t3toolbox/backend/orthogonal_representations.py`

Now `gauge_residual` in `tangent_operations.py`:

▸ **Read** `t3toolbox/backend/tangent_operations.py`

▸ **Edit** `t3toolbox/backend/tangent_operations.py`

Now the frontend checkers — the residuals now return `xnp` scalars, so I'll wrap the threshold comparisons in `bool()` to preserve the clean Python-`bool` contract (eager-only; checkers aren't jitted):

▸ **Edit** `t3toolbox/basis_variations_format.py`

▸ **Edit** `t3toolbox/basis_variations_format.py`

▸ **Edit** `t3toolbox/manifold.py`

Now `t3_linalg:188 np.ones` → `xnp.ones`:

▸ **Read** `t3toolbox/backend/t3_linalg.py`

▸ **Edit** `t3toolbox/backend/t3_linalg.py`

Now the `np.prod` → `math.prod` style cleanup (on the recorded list for `bv_operations`; same pattern in the `.size` properties). Let me check which files need import `math` and read the exact lines:

▸ **Bash** — Check import `math` and import blocks

All four files need `import math`. Let me read the import regions for `bvf` and `ttt` (I have manifold's and `bv_operations`):

- ▷ **Read** `t3toolbox/basis_variations_format.py`
- ▷ **Read** `t3toolbox/tucker_tensor_train.py`

Now adding `import math` and changing `np.prod` → `math.prod` across the four files:

- ▷ **Edit** `t3toolbox/backend/bv_operations.py`
- ▷ **Edit** `t3toolbox/backend/bv_operations.py`
- ▷ **Edit** `t3toolbox/manifold.py`
- ▷ **Edit** `t3toolbox/manifold.py`
- ▷ **Edit** `t3toolbox/basis_variations_format.py`
- ▷ **Edit** `t3toolbox/basis_variations_format.py`
- ▷ **Edit** `t3toolbox/tucker_tensor_train.py`
- ▷ **Edit** `t3toolbox/tucker_tensor_train.py`

Now add `test_dispatch` coverage for the now-dispatching functions: the three residuals (jit bucket) and `TuckerTensorTrain.inner` (covers the `t3_linalg.xnp.ones` fix, which wasn't tested before):

- ▷ **Edit** `tests/test_dispatch.py`
- ▷ **Edit** `tests/test_dispatch.py`
- ▷ **Edit** `tests/test_dispatch.py`

All edits done. Let me run the dispatch suite first (the key validation that the residuals now jit cleanly with no numpy leak), then the full suite:

- ▷ **Bash** — Run dispatch suite after raw-np fixes

Dispatch suite passes — the residuals now jit cleanly. Now the full suite:

- ▷ **Bash** — Run full relevant suite after dispatch fixes

All 150 pass. Let me confirm no doctest regressions on the touched modules (the residual rewrites + `bool()` wraps could affect captured `True/False` outputs), and do a quick numpy-vs-jax numerical agreement check on the rewritten residuals:

- ▷ **Bash** — Numerical agreement + A/B doctest counts

Everything verified:

- `numpy` ≡ `jax` residuals (orth $1.33e-15$, gauge 2.72, consistency agree);
- `is_orthogonal()` now returns a true Python `bool` (the wrap works), checkers behave correctly;
- **A/B doctest: 211 = 211 — zero new doctest failures.**

Now let me mark follow-up (A) resolved in the refactor doc. Let me read the current Follow-ups section:

- ▷ **Read** `docs/backend_separation_refactor.md`
- ▷ **Edit** `docs/backend_separation_refactor.md`

Follow-up (A) marked resolved. Let me stage exactly the touched files and commit:

- ▷ **Bash** — Confirm touched files before staging
- ▷ **Bash** — Commit and push the dispatch fixes

Committed and pushed (b07759b4).

Dispatch bug list — done

I swept all validated modules + verified backend deps for raw `np.*`, triaged each, and fixed the genuine dispatch bugs:

Fixed (genuine dispatch bugs):

- **3 residual/checker backends** — `basis_orthogonality_residual`, `basis_consistency_residual` (`orthogonal_representations`), `gauge_residual` (`tangent_operations`): rewritten to dispatch via `xnp` and be **jit-able** (per-core scalar devs → `xnp.stack` → `xnp.max`; dropped the `float()/Python-max()` that coerced to numpy and broke jit). Frontend checkers wrap the threshold in `bool()` for a clean Python-bool contract. **numpy ≡ jax verified**; added to the `test_dispatch_jit` bucket.
- **t3_linalg** inner-product zipper: `np.ones` → `xnp.ones`, plus a `TuckerTensorTrain.inner` jit test (that path was previously uncovered).
- **np.prod** → **math.prod** (house style) in `variations_from_vector` + the four `.size` properties.

Triaged and left numpy (not dispatch bugs): `save_core_families/load_core_families` (file I/O is always concrete numpy), `ranks.py np.minimum` (integer rank arithmetic, no array data), `t3_operations np.broadcast_shapes` (tuple shape arithmetic), `unit_variations np.zeros` (pure constructor → `to_jax`).

Verification: dispatch suite green (residuals now jit cleanly = no numpy leak); 150 tests pass; A/B doctest **211 = 211** (zero new failures); `is_orthogonal()` confirmed returning a real Python bool. Follow-up (A) marked ✓ **RESOLVED** in `docs/backend_separation_refactor.md`.

Still open / on the radar

- **Follow-up (B)** — the sum-axis no-wrap convention (`corewise_stack_sum`); flagged “discuss after the refactor.” Quick yes/no when you want.
- **Doctest hygiene** — ~190 stale failures in `tucker_tensor_train.py` (mostly module-level `t3.t3_corewise_randn`-style references that no longer exist; the now-fixed save/load ones are gone). A separate sweep if you want it.

What next — (B), the doctest hygiene sweep, or more of your review findings?

Nick

Ok, great! Let’s do something else before we get to those tasks. This is more mathematical/algorithmic; I think you will find it fun. Currently, orthogonalization of a dense tensor onto the tangent space is done by performing `t3svd` of the dense tensor, then projecting the resulting enormous T3 onto the tangent space. This reuses exiting functions, but is computationally expensive because of the large SVDS. I want to talk through a different algorithm for this projection which only needs contractions. I have worked out this algorithm, but not implemented it. I am pretty certain it is right, but we need to make sure. You will check my work by reasoning, then by numerical tests. Then, when we are sure it is right, we can implement it. To prepare, please read `project_dense_onto_tangent` in `manifold.py`

Claude

Looking forward to this one. Let me read `project_dense_onto_tangent` and the projection machinery it relies on, so I’m ready to check your algorithm.

▷ **Read** `t3toolbox/manifold.py`

Now `T3Tangent.project` and the backend it delegates to:

▷ **Read** `t3toolbox/manifold.py`

Now the backend projection — `project_t3_onto_tangent_space`:

▷ **Bash** — Locate `project_t3_onto_tangent_space`

▷ **Read** `t3toolbox/backend/tangent_operations.py`

Let me read `tangent_to_dense` and the zipper functions — these define the tangent-space frames, which is exactly the geometry I’ll need to check your algorithm against.

▷ **Read** `t3toolbox/backend/tangent_operations.py`

Let me read the two zipper functions (the environment builders) to complete my picture of the frames:

▷ **Bash** — Locate zipper functions

▷ **Read** `t3toolbox/backend/tangent_operations.py`

I've read `project_dense_onto_tangent` and traced the whole projection machinery it (indirectly) relies on. I'm ready. Here's my mental model, so you can correct me before we start — getting the conventions right matters for checking your algorithm.

Current approach

```
x, _, _ = t3svd_dense(dense_tensor)      # dense Z -> exact T3 (huge ranks)
return T3Tangent.project(x, basis)        # project that T3 onto the tangent space
```

`T3Tangent.project` → `project_t3_onto_tangent_space(basis, x)`, which contracts **x's cores** against the base frames (re-express x's TT in the base Tucker basis, build left/right **zipper**s = environments, form dG/dB per core, then `orthogonal_gauge_projection`). The cost is entirely in `t3svd_dense` — SVDs of the dense tensor's 2d-1 unfoldings, with the intermediate T3 carrying full (exponentially large) ranks. The projection itself is already just contractions.

The geometry I'll be checking against (Appendix A.3)

The orthogonal T3Basis gives four orthonormal core families:

- U_i (up_tucker): $n_i \times N_i$, **row-orthonormal** ($U_i U_i^T = I_{\{n_i\}}$, from `einsum('...io,...jo->...ij')`) — the Tucker mode factors.
- L_i (left): **left-canonical** TT cores ($\sum_{\{i,a\}} L[i,a,j] L[i,a,k] = \delta_{\{jk\}}$).
- R_i (right): **right-canonical** TT cores.
- $O_i = D_i$ (down/outer): outer-orthonormal ($\sum_{\{i,j\}} D[i,a,j] D[i,b,j] = \delta_{\{ab\}}$) — the orthonormal frame for the Tucker-variation/central-TT slot.

Tangent space T = span of 2d single-core replacements: d Tucker-variation directions (δU_i , shape $n_i \times N_i$) and d TT-variation directions (δG_i , shape $r_i \times n_i \times r_{\{i+1\}}$). `project` returns the orthogonal projector **onto the linear subspace T** (it does *not* subtract the base point; identity on T).

Why contraction-only should work — and what I'll watch for

The projection coefficients are inner products of Z with **orthonormal frames built from the base cores** (left env from $U_1..U_{\{i-1\}}+L$, the slot U_i/O_i , right env from $U_{\{i+1\}}..U_d+R$). So you can contract Z directly against those frames instead of SVD-compressing it first. The three things I'll scrutinize:

1. **Orthonormality of the frames** — that the left/right environments built from $L/R/U$ are genuinely orthonormal maps, so a bare contraction gives the *orthogonal*-projection coefficient (not an oblique one).
2. **The gauge** — the current code produces *ungauged* dG/dB then calls `orthogonal_gauge_projection`. I'll check whether your contractions land gauged directly or still need that step.
3. **No double-counting / cross-terms** between the 2d directions (the reason the existing code gauges).

One question before you walk me through it: should I assume the same precondition as the current code — **orthogonal, minimal-rank basis** — and that you want the result in the same gauged (`tucker_variations`, `tt_variations`) form `project` returns?

Go ahead and lay out your algorithm — I'll check the reasoning first, then we'll nail it down numerically.

Nick

Yes, assume orthogonal minimal-rank basis and gauged output. And I think you are already predicting the method. To get one component of the variation for a tangent vector, we contract the dense tensor against the frame surrounding that variation core. This results in a core with the shape of the variation,

but is ungauged. To make it gauged, we apply the orthogonal gauge projection. The resulting combination (contraction with orthogonal frame then orthogonal gauge projection), applied to all variation cores, yields the orthogonal projection of the dense tensor onto the tangent space. The numerical check is to assemble a dense orthogonal basis for the tangent space by forming unit tangent vectors, converting them to dense, then orthogonalizing them. Then the dense tensor can be projected onto the dense tangent space using dense linear algebra. We compare this to applying the projection algorithm, then doing `to_dense()` on the resulting tangent vector. I am certain that the orthogonality condition is required for the frame. I am not certain whether the minimal rank condition is required.

Claude

I've checked your algorithm by reasoning, and I believe it's **correct**. Let me show the derivation, then confirm numerically — including a direct test of the minimal-rank question, which (interestingly) the codebase itself is currently inconsistent about (the project docstring says minimal-rank is required, but `CLAUDE.md` claims "project works on any orthogonal base").

Reasoning

Step 1 — frame contraction gives the right ungauged core. For TT-slot i , the surrounding frame is $F_i = \Phi^L_i \otimes U_i \otimes \Phi^R_i$, where $\Phi^L_i[r_i; x_{\{1..i-1\}}]$ is the left interface built from $U_{\{1..i-1\}} + L_{\{1..i-1\}}$. Contracting Φ^L_i with itself over the left modes: each $\sum_{x_j} U_j[n_j, x_j] U_j[n'_j, x_j] = \delta$ (U row-orthonormal) collapses $n_j = n'_j$, then the left-canonical condition $\sum_{r_j, n_j} L_j[r_j, n_j] L_j[r'_j, n_j] = \delta$ propagates $r_i = r'_i$ from the boundary. So Φ^L_i is **orthonormal** — and likewise Φ^R_i (right-canonical) and U_i . Hence F_i is an isometry, so $\langle F_i, Z \rangle$ is exactly the orthogonal-projection coefficient onto $\text{image}(F_i)$. The Tucker frame W_i is orthonormal over n_i by the outer-orthonormality of 0_i . **This step needs the orthogonality (canonical) conditions, full stop.**

Step 2 — why gauge-then-sum equals the orthogonal projection. The ungauged frame images overlap (every $\text{image}(F_i)$ contains the base point), so naively $\sum_i P_{\{\text{image}_i\}} \neq P_T$. The gauge fixes this by making the per-direction subspaces **mutually orthogonal**. The key cross-term, for gauged $\delta G_i \perp L_i$ and any δG_j with $i < j$:

$$\begin{aligned} \langle T_i, T_j \rangle &\rightarrow (\text{collapse modes } 1..i-1 \text{ via } \Phi^L_i \text{ orthonormality} \rightarrow \delta_{\{r_i, r'_i\}}) \\ &\rightarrow (\text{collapse mode } i \text{ via } U_i U_i^T = \delta \rightarrow n_i = m_i) \\ &\rightarrow \sum_{r_i, n_i} \delta G_i[r_i, n_i, \cdot] \cdot L_i[r_i, n_i, \cdot] = 0 \quad \leftarrow \text{exactly the gauge } \delta G_i \perp L_i \\ &\hookrightarrow L_i \end{aligned}$$

The U-U cross-terms vanish by $\delta U_i \perp U_i$, and every U-G cross-term vanishes because the U-direction's open mode x_i always meets a U_i in the G-direction, giving $\delta U_i U_i^T = 0$. **All of these use only canonical orthogonality + the gauge — no minimality.** And $\text{span}(\text{ungauged}) = \text{span}(\text{gauged})$ by the nesting identity $\Phi^R_i = (U_{\{i+1\}} R_{\{i+1\}}) \Phi^R_{\{i+1\}}$ (the L_i -parallel part of $\text{image}(F_i)$ is absorbed into $\text{image}(F_{\{i+1\}})$), again purely canonical.

Since F_i is an isometry, $F_i(P_{\perp L_i} F_i^* Z) = P_{\{\text{gauged-image}_i\}}(Z)$, and the gauged images are orthogonal with direct sum = T . Therefore $\text{to_dense}(\text{gauged result}) = \sum_i P_{\{\text{gauged-image}_i\}}(Z) = P_T(Z)$. ■

Conclusions: the algorithm is correct; **orthogonality is required** (both for frame orthonormality and for the cross-terms to cancel); ****minimal rank is not required**** — every step used only the canonical conditions. My prediction: it'll match the dense projection on a non-minimal orthogonal basis too.

Now let me confirm all of this numerically.

▷ **Bash** — Numerical check: minimal orthogonal, no-gauge, non-orthogonal

Strong initial results: **algorithm matches to 1.35e-15**, and the two negative controls fail as predicted (no-gauge: 1.82; non-orthogonal: 5.04). Interestingly, `random_orthogonal((3,3,2), (1,2,2,1))` already reports `has_minimal_ranks=False` yet passes — let me understand the rank situation and test clearly-minimal vs clearly-non-minimal (padded) bases to settle the minimality question rigorously.

▷ **Bash** — Diagnose ranks and test minimal vs padded non-minimal bases

Decisive. The minimal-rank question is settled:

Case	relerr	orthogonal	minimal	result
minimal-rank basis	1.37e-15	✓	✓	match
non-minimal (padded) basis	9.83e-16	✓	✗	match
no gauge	1.82	✓	—	fails (gauge needed)
non-orthogonal	5.04	✗	—	fails (orthogonality needed)

The padded basis is a genuine rank-(2,2,2)/(1,2,2,1) tensor inflated to nominal (3,4,2)/(1,3,3,1), re-orthogonalized — has_minimal_ranks=False, represents the **same** tensor (norm diff 0.0), and the algorithm still nails the projection to 9.8e-16. So **minimal rank is not required** — exactly as the reasoning predicted, and confirming CLAUDE.md's "project works on any orthogonal base" over the project/project_dense docstrings' "requires minimal-rank" (those are too strong).

Let me run one robustness sweep across more shapes/dims (incl. d=4) to be fully sure before we implement.

▷ **Bash** — Robustness sweep across shapes/dims, minimal and non-minimal

All six configurations match to $\sim 1e-15$ (d=3 and d=4, minimal and non-minimal, two seeds each). Combined with the two negative controls failing exactly as predicted, the algorithm is confirmed.

Verdict: your algorithm is correct ✓

Both the reasoning and the numerics agree completely:

- **Correct.** (contract Z against each variation's orthonormal frame) → orthogonal_gauge_projection reproduces the dense orthogonal projection $P_T(Z)$ to machine precision.
- **Orthogonality is required** — and for *two* reasons, not one: (a) the frames must be orthonormal for a bare contraction to give the orthogonal (not oblique) coefficient, and (b) the cross-terms between directions only cancel because the gauge condition meets a *canonical* core ($\delta G_i \perp L_i$, $\delta U_i \perp U_i$). The non-orthogonal control fails (relerr 5.0).
- **The gauge is essential** — without it you'd sum overlapping projections and overshoot (relerr 1.8). It's what orthogonalizes the 2d directions so "sum of per-direction projections = projection onto their span."
- **Minimal rank is NOT required.** Every step of the derivation used only the canonical conditions, and the padded non-minimal basis confirms it (relerr 9.8e-16). This resolves the codebase's open question: **CLAUDE.md is right ("any orthogonal base"), the project/project_dense_onto_tangent docstrings are wrong** ("requires minimal-rank" is too strong — worth fixing when we touch them).

One honest caveat about *what* the test proved: my prototype computes the ungauged cores as $(\text{unit_tangent}_\alpha, Z)$ (densify each unit, dot with Z). That is *mathematically* exactly "contract Z against the frame," so it validates the **math** — but it's $O(2d \cdot \text{core_entries} \cdot N^d)$ and not the efficient algorithm. The non-circular part it checks is precisely your claim (the gauge G correcting the frame overlap $A^T A \neq I$), so the math is solid.

Implementation path (for when you're ready)

The efficient version mirrors the existing project_t3_onto_tangent_space (zippers + per-core envs + gauge), but the left/right environments are built by **absorbing Z's modes one at a time against the base chains** (U_j, L_j) (left) and (U_j, R_j) (right) — reducing each mode $N_j \rightarrow n_j \rightarrow \text{bond}$ — rather than zipping two TT chains. Each slot i then contracts its left env, U_i , and right env into a small (r_i, n_i, r_{i+1}) core; the Tucker cores use the 0_i env. Then the same orthogonal_gauge_projection. No SVDs, and no giant intermediate T3.

Two things to decide before I draft it:

1. **Where it lives / API** — replace the body of project_dense_onto_tangent (keep the signature), with the contraction core as a new backend fn like project_dense_onto_tangent_space(basis, Z) in tangent_operations.py (parallel to project_t3_onto_tangent_space)?
2. **Verification harness** — I'd keep the dense-projector reference (unit vectors → orthonormalize → QQ^T) as the test oracle, plus check the new path agrees with the *current* t3svd-then-project path. Good?

Want me to draft the efficient backend implementation, or would you like to walk through the contraction/environment structure together first?

Nick

Excellent! For 1, I would like to retain both methods, and allow the user to switch between them with an option. The default should be the new contraction-based method. Please make a function for the new method in the backend in parallel to the existing method. 2. You can check against the dense projector reference as the test. No need to compare to the existing t3svd method. If you feel confident about this, you can begin. Please keep in mind the library conventions: frontend vs backend, xnp, and stacking.

Claude

Confirmed and approved — I'll implement it. Given the index bookkeeping is intricate, I'll work in stages: first nail the efficient contraction in a numpy prototype and verify it against the dense reference, *then* write the production xnp/stacking-aware backend function, wire the frontend option, and add tests.

First, a key simplification I worked out: both variations at slot i come from the **same** $\text{core_env}_i[r_i, x_i, r_{i+1}]$ (Z reduced over every mode except i): $\delta G_i = \langle U_i, \text{core_env}_i \rangle$ and $\delta U_i = \langle 0_i, \text{core_env}_i \rangle$. Let me prototype and verify that.

▷ **Bash** — Numpy prototype of efficient contraction algorithm vs dense reference

O bonds match L/R bonds (confirmed: (1,2,2) for both). Just a variable-scope bug — I referenced basis inside the function. Let me fix and rerun:

▷ **Bash** — Fixed prototype vs dense reference, minimal and non-minimal

The efficient contraction prototype matches the dense reference to $\sim 1e-15$ across $d=3$, $d=4$, minimal and non-minimal. The math and my implementation structure (left sweep + per-slot right reduction $\rightarrow$ shared $\text{core_env} \rightarrow \delta G, \delta U \rightarrow$ gauge) are confirmed.

Now let me write the production version with xnp + stacking and test it standalone (unstacked, stacked, numpy, jax) before wiring it in:

▷ **Bash** — Stacked + jax + jit test of production-style function

Excellent — the production-style function is correct across **all** dimensions of variation: unstacked numpy ($1.6e-15$), jax ($1.7e-15$, output jax), stacked $C=(2,)$ (per-slice $\sim 1e-15$), and **jit-compiles** with all-jax output. Now let me wire it into the codebase. First, locate where to place the backend function:

▷ **Bash** — Find function boundaries for placement

▷ **Read** t3toolbox/backend/tangent_operations.py

Confirmed correct in every mode. Now I'll wire it in. First, add import math and the backend function to tangent_operations.py:

▷ **Read** t3toolbox/backend/tangent_operations.py

▷ **Edit** t3toolbox/backend/tangent_operations.py

▷ **Edit** t3toolbox/backend/tangent_operations.py

▷ **Edit** t3toolbox/backend/tangent_operations.py

Now the frontend — add the method option (default 'contraction'), retaining 't3svd', and fix the docstring (orthogonal, not minimal-rank):

▷ **Edit** t3toolbox/manifold.py

Now let me fix the same "minimal-rank" overclaim in the two sibling project docstrings (we've established it's not required):

▷ **Edit** t3toolbox/manifold.py

▷ **Edit** t3toolbox/backend/tangent_operations.py

Now let me verify the wired-in frontend works (both methods, default = contraction) and agrees with the dense reference:

▷ **Bash** — Verify wired frontend: both methods, default, riemannian_gradient, error, project_t3 non-...

All wired up and working: default (contraction) matches the reference to $1.6e-15$, both methods agree, riemannian_gradient uses the new path, bad method raises, and gauged output is confirmed. Now let me add proper tests. Let me look at test_manifold.py's conventions to add the numerical test:

▷ **Bash** — Survey test_manifold.py structure

There's already a _dense_tangent_projector helper. Let me read it and find the existing projection test to extend:

▷ **Read** tests/test_manifold.py

Let me find the existing test that uses _dense_tangent_projector (likely the current project test):

▷ **Bash** — Find existing project tests

There's already a test_project_dense_onto_tangent (line 716) comparing against _dense_tangent_projector — and since the default is now 'contraction', it **already covers the new method**. Let me read it to see what to add (both-methods + non-minimal):

▷ **Read** tests/test_manifold.py

The existing test_project_dense_onto_tangent already exercises the contraction path (it's now the default). Let me extend it to cover **both methods explicitly** and a **non-minimal basis** (locking in the minimal-not-required result), then add a jit dispatch case.

▷ **Edit** tests/test_manifold.py

Now the jit dispatch case in test_dispatch.py (the contraction method has static shapes, so it's jit-able):

▷ **Edit** tests/test_dispatch.py

Let me run the affected suites first, then the full suite:

▷ **Bash** — Run manifold and dispatch suites

▷ **Bash** — Run full relevant suite

All 150 pass. Let me confirm no doctest regressions on the two touched modules, then commit:

▷ **Bash** — A/B doctest check on touched modules

No doctest regressions. Let me commit the new feature.

▷ **Bash** — Confirm touched files

▷ **Bash** — Commit and push the contraction projection

Done — committed and pushed (10a540ea).

Summary

The contraction-only dense tangent projection is implemented, verified, and shipped.

Verification path (reasoning → numerics, as you wanted):

- Reasoned it correct: contract-against-orthonormal-frame + gauge = the dense orthogonal projector. Established that **orthogonality is required** (frame isometry *and* gauge cross-term cancellation) but **minimal rank is not** — every step uses only the canonical conditions.
- Confirmed numerically against the dense-projector reference to $\sim 1e-15$ across $d=3/d=4$, minimal and **non-minimal** bases, multiple seeds — with the negative controls (no-gauge, non-orthogonal) failing exactly as predicted.

What landed:

- **Backend** `tangent_operations.project_dense_onto_tangent_space(basis, Z)` — the efficient algorithm. Key simplification: both variations at mode i read off a **shared** `core_env_i = Z` reduced over every other mode ($\delta G_i = \langle U_i, \text{core_env}_i \rangle, \delta U_i = \langle 0_i, \text{core_env}_i \rangle$). One left sweep + per-slot right reduction + gauge. xnp/stacking-aware, jit-able (static shapes, no SVD).
- **Frontend** `project_dense_onto_tangent(..., method=...)` — 'contraction' (default) / 't3svd' (retained). `riemannian_gradient` inherits the new default.
- **Docstrings corrected**: `project` / `project_t3` / `project_dense` no longer claim "minimal-rank required" (resolves the open question — CLAUDE.md's "any orthogonal base" was right).
- **Tests**: extended `test_project_dense_onto_tangent` (both methods agree vs the dense projector; non-minimal base; bad-method raises) + a jit dispatch case. 150 green, no doctest regressions.

That was a fun one — the shared-core_env structure made it come out cleaner than I expected.

Back on the radar whenever you want them: follow-up **(B)** (the sum-axis no-wrap convention) and the **doctest hygiene** sweep in `tucker_tensor_train.py`. Or wherever you'd like to go next.

Nick

Ok great! There is one small thing I noticed. The library convention is to put the function arguments to be in their own lines (within reason). The reason for this is so that each argument, and the return type, can have a comment after it which explains shapes. For examples of this, look at the functions in `probing.py`. Can you please update the function signature to use this convention, and make a note to yourself about this convention?

Claude

Let me look at `probing.py`'s exact signature style (alignment, comment placement) so I match it precisely.

▷ **Read** `t3toolbox/backend/probing.py`

Got it — and you're right, I applied the one-arg-per-line convention to the new backend function but wrote the **frontend** `project_dense_onto_tangent` inline. Let me fix that signature:

▷ **Edit** `t3toolbox/manifold.py`

Now let me record the note about this convention. Writing a feedback memory:

▷ **Write** `/home/nick/.claude/projects/-home-nick-repos-T3Toolbox/memory/signature-args-one-per-line.md`

▷ **Read** `/home/nick/.claude/projects/-home-nick-repos-T3Toolbox/memory/MEMORY.md`

▷ **Edit** `/home/nick/.claude/projects/-home-nick-repos-T3Toolbox/memory/MEMORY.md`

Let me verify the reformatted signature parses and works, then commit:

▷ **Bash** — Verify signature parses and manifold tests pass

Signature parses correctly and tests pass. Committing the formatting fix (the memory lives in `~/ .claude`, outside the repo):

▷ **Bash** — Commit the signature formatting fix

Done — committed and pushed (3ffeccc7).

- **Reformatted** `project_dense_onto_tangent`'s signature to one-arg-per-line with aligned trailing shape comments and a commented return type, matching the `probing.py` convention. (The backend `project_dense_onto_tangent_space` already followed it.)
- **Noted the convention** as a feedback memory (`signature-args-one-per-line.md`, indexed in `MEMORY.md`): each arg on its own line with an aligned shape comment so every argument and return element can document its shape — and crucially, **this applies to frontend functions too**, which is exactly where I slipped.

Verified the signature still parses and the manifold suite is green.

Nick

Excellent! Now there is another thing I would like to talk about regarding `project_dense_onto_tangent`. The contraction method is not a total win, because there is a regime in which the `t3svd` method can outperform. That is when the `t3svd` performs intermediate rank truncations during the sweeping svd process, using `rtol`, `atol`, or maximum ranks. I would like your opinion on whether it is a good idea to add these options to `project_dense_onto_tangent`. If so, we have to answer the following questions: (a) how does the user specify these options (I would say optional kwargs)? (b) How do we test correctness, given that orthogonality will no longer hold to machine precision?

Claude

Good question — this is a real design call, and I want to give you an informed opinion. Let me first check how `t3svd_dense` does its SVDs (truncated vs full), since that affects when truncation actually wins.

- ▷ **Bash** — Check `dense_t3svd` SVD strategy
- ▷ **Bash** — Check `truncated_svd` implementation (full vs randomized)
- ▷ **Bash** — Check if `truncated_svd` does full SVD then slice
- ▷ **Bash** — Read `truncated_svd` body

I dug into `truncated_svd` before answering, and it changes my opinion — let me lead with that, because it bears directly on whether this is worth doing.

The cost reality (with the current implementation)

`linalg.truncated_svd` does a **full** `xnp.linalg.svd(..., full_matrices=False)` and then *slices* to rank r . So for `t3svd_dense`, the first unfolding SVD (matricize Z as $N_0 \times N_1 \cdots N_{\{d-1\}}$) costs $O(N^d \cdot N_0)$ **regardless of any `rtol/atol/max-rank`** — truncation can't avoid the full SVD, it only shrinks the *later* sweeps and the final `project(x̃)`.

Compare to the contraction method: its dominant step is the first mode-absorption, $O(N^d \cdot n)$ where n is the **Tucker rank**. Since $n \leq N_0$, the contraction's bottleneck is already $\leq$ `t3svd`'s *first SVD alone* — before truncation can help anything. For a compressible Tucker ($n \ll N$) the gap is large ($O(N^d \cdot n)$ vs $O(N^d \cdot N)$).

So with the **current** full-SVD path, I don't see a regime where `t3svd-truncated` beats contraction — truncation only speeds up steps that aren't the bottleneck. The crossover you're describing seems to require a **randomized / partial SVD** (top- k , $O(N^d \cdot k)$), so that the *first* unfolding also gets cheaper, **and** you accept a coarse projection ($k < \text{base rank}$, tolerating $\sim \text{rtol}$ error).

So my real question back: are you anticipating a randomized SVD (or have a regime in mind I'm missing)? If yes, the feature makes sense and I'd add it. If we're staying with full SVDs, I'd lean *hold off* — it adds API surface + a testing burden without a speed win, since contraction dominates it everywhere.

That said — the design answers are clean either way, so here they are:

(a) How the user specifies the options

Optional kwargs that mirror `t3svd_dense`, **only meaningful for `method='t3svd'`**:

```
def project_dense_onto_tangent(
    dense_tensor, basis, method='contraction', *,
    rtol=None, atol=None, max_tucker_ranks=None, max_tt_ranks=None):
```

Guard: if any truncation kwarg is set with `method='contraction'`, **raise** (contraction is exact — it has no rank to truncate). They pass straight through to `t3svd_dense`, so its existing constraints (e.g. `rtol/atol` require unstacked) propagate for free. I'd keep the explicit `method='t3svd'` requirement rather than auto-switching, so the speed/accuracy tradeoff is a deliberate choice.

(b) How to test correctness without machine-precision orthogonality

Two complementary checks — together they pin it down rigorously:

1. **The projection part is still exact (machine precision).** With truncation the method computes $P_T(\tilde{x})$ for the truncated $\tilde{x}$. Test that it equals the contraction projection of that *same* $\tilde{x}$ densified:

```
project(Z, 't3svd', rtol=r) ≈ project( t3svd_dense(Z, rtol=r).to_dense(), 'contraction' )  
↪ # ~1e-15
```

Both compute $P_T(\tilde{x})$ for the same $\tilde{x}$, so they agree to machine precision — this confirms truncation + projection compose correctly.

1. **The approximation is controlled (the real new behavior).** Since P_T is a non-expansive orthogonal projection, $\|P_T(Z) - P_T(\tilde{x})\| = \|P_T(Z - \tilde{x})\| \leq \|Z - \tilde{x}\|$. So compute the exact projection (contraction) and the *computable* truncation error, and assert:

```
|| project(Z, 'contraction').to_dense() - project(Z, 't3svd', rtol=r).to_dense() ||  
≤ || Z - t3svd_dense(Z, rtol=r).to_dense() || (+ a small slack)
```

This is the answer to “orthogonality no longer holds to machine precision” — we don’t test exact equality, we test that the projection error is bounded by the truncation error, which is exactly the guarantee truncation should give.

What regime did you have in mind for the crossover — is a randomized SVD on the roadmap?

Nick

Oh, I see. Hmm. The reason I thought the t3svd method might be good was because it is written in an influential paper by Steinlechner (for tensor trains, no tucker) from about 10 years ago. I assumed there must be a good reason, and latched onto the truncation idea, without thinking through carefully. Can we definitively rule out the t3svd approach? Why might they have presented it in the paper?

Claude

Good instinct to interrogate this rather than cargo-cult it — and your intuition that “the SVD must matter here” is actually **correct**. You just attached it to the wrong operation.

Short answer: yes, rule it out as *the dense-projection method*

For projecting a genuinely dense tensor onto the tangent space, contraction strictly dominates: it’s exact, and its bottleneck $O(N^d \cdot n)$ is already below t3svd’s *first full SVD* $O(N^d \cdot N)$ — and truncation can’t touch that first SVD. There’s no regime (with full SVDs) where t3svd-then-project wins. So: contraction is the right default (done), and the truncation knob isn’t worth adding.

I’d still **keep the t3svd option**, but for one honest reason: it’s an *independent implementation* of the same map, which is exactly what cross-checked our contraction code (test_project_dense_onto_tangent compares them). That’s worth having as a fallback/oracle — just not as a performance path.

Why the SVD is all over that literature — projection vs. retraction

In Riemannian optimization on the fixed-rank manifold there are **two** distinct operations, and they use different tools:

1. **Projection** P_T (gradient $\rightarrow$ tangent space). In Holtz-Rohwedder-Schneider, Lubich-Oseledets-Vandereycken, and Steinlechner, this is the **orthogonal-decomposition formula** $P_T = \sum_i P_i$, evaluated by **left/right interface contractions** — i.e. it’s the *contraction* method. The papers don’t advocate “SVD the input, then project”; the projection itself is contractions.
2. **Retraction** (tangent vector $\rightarrow$ back onto the manifold). This is where **truncated TT-SVD is load-bearing**: you form the rank-doubled embedding base + v and truncate it back to the base ranks. That truncation *is* a truncated SVD, and rtol/atol/max-rank are exactly the right knobs there. We already do this — T3Tangent.retract calls t3svd with max ranks = base ranks. Plus TT-SVD is

the canonical tool for **rounding** and for building the **canonical/orthogonal representation** in the first place.

So your instinct ("there must be a good reason for the SVD") is right — it's just the reason for the *retraction* and *rounding*, not the projection.

The deeper reason it's a non-issue in that setting

In the high-d regime where TT/T3 methods exist, **you never materialize a full dense N^d tensor** — it's exponentially large. The thing you project is always *structured*: in Steinlechner's tensor completion the Euclidean gradient is **sparse** (supported on the observed entries), and his real efficiency contribution is projecting *that* by contracting only over the nonzeros. If your input is already a TT, you project it with `project_t3` directly. "Project a dense tensor" is essentially a convenience / small-case / testing operation — and for *that*, the contraction is the correct tool, while converting dense→TT→project is just the lazy code-reuse path (which is literally why this library had it).

So nothing in the literature is in tension with our conclusion: the SVD belongs to retraction/rounding/canonicalization and to structured inputs; the dense projection is a contraction.

One caveat on intellectual honesty: I'm reasoning from the general structure of that body of work, not a fresh re-read of Steinlechner's exact text. If you want, I can pull the paper (it's on arXiv) and confirm precisely where he uses TT-SVD vs. contraction — but I'd bet it's projection-by-contraction, retraction-by-truncated-SVD.

Net recommendation: **keep both methods, contraction as default, skip the truncation options.** Want me to leave it exactly as-is, or add a one-line note in the `project_dense_onto_tangent` docstring explaining that 't3svd' exists as an independent cross-check and that truncation isn't offered because contraction dominates?

Nick

Ok, this makes sense. I understand. Let's keep both methods, contraction as default, skip the truncation options. We can leave the docstring as is. Now I would like to move on to another mathematical/algorithmic method, which is the TTM algorithm for elementwise multiplication of two tensor trains. I want you to read the paper for tensor trains and understand it. We will then discuss its generalization to Tucker tensor trains, which I believe is straightforward. Once this is clear, we can make a plan for implementation. I have placed it as TTM.pdf in the project root directory. Do not add this paper to git; it is just for your reference here. Some things to keep in mind: 1) this method can take advantage of truncation. 2) we already have elementwise multiplication via a more conventional method, so this would be an extra method for the same operation.

Claude

Let me locate the paper and check its length before reading.

▷ **Bash** — Check TTM.pdf existence and page count

14 pages. Let me read it.

▷ **Read** TTM.pdf

This is the core algorithm. Let me read the rest (numerical results + appendices, especially the toy-model analysis of intermediate bond growth).

▷ **Read** TTM.pdf

I've read the whole paper (algorithm in §II B / Fig 1, scaling and the bond-blowup caveat in §III + Appendix A). Here's my understanding, then how I think it maps onto T3 — and the one structural question I'd want us to align on before planning.

TTM (the TT case), as I understand it

Goal: elementwise product $\Omega = \psi \circ \phi$ of two TTs, length N , bonds χ, χ' .

Conventional: promote ϕ to an MPO by hanging a local COPY tensor ($\delta_{ijk}^0 = 1$ iff $i=j=k$) on each physical leg, then contract with ψ . The contracted carriage is an **outer product** $\Pi^\sigma = K^\sigma \otimes M^\sigma$, so the bond jumps to $\chi\chi'$; you then variationally compress. Cost $O(\chi^4)$, memory $O(\chi^3)$.

TTM: never form the MPO / the $\chi\chi'$ bond. Instead:

1. **Concatenate** the two TTs into one length- $2N$ TT, with ψ **reversed** ($K^{\{\sigma_1\} \dots \{\sigma_N\}} M^{\{\omega_N\} \dots \{\omega_1\}}$) so the index- N carriages start adjacent and the total swap count is minimized.
2. **Contract** (eq 9): when the two same-index carriages are adjacent, fuse them with the COPY tensor ($\delta_{\{\sigma_i \omega_i \eta_i\}}$ merges the two physical legs) + the shared-bond contraction $\rightarrow$ one output carriage. $O(\chi^3)$.
3. **Swap** (Fig 1C): to bring same-index carriages together — contract the pair, permute physical legs, reshape, **SVD, and truncate** to threshold ϵ (drop the small singular values, $\|\psi - \psi'\| \leq \sqrt{\epsilon} \|\psi\|$), moving the gauge center rightward. Mixed-canonical gauge keeps the truncation quasi-optimal; QR (not SVD) moves the gauge after a contraction.
4. Iterate; carriage i of ψ swaps $i-1$ times.

Payoff: $O(\chi^3 N^2)$ ops, $O(\chi^2)$ memory — quartic $\rightarrow$ cubic, and $\chi^3 \rightarrow \chi^2$ memory. **The truncation ϵ is the knob** (your point 1): it's load-bearing because the swaps turn short-range into long-range correlations, so **intermediate bonds blow up** (Fig 3C/D: peak $\approx 2-5\chi$, depending on ϵ and the singular-value decay of the inputs). That intermediate peak is the real bottleneck, and the crossover vs. conventional sits at $\chi \approx 200-1400$ depending on ϵ .

The T3 elementwise product — structure

Writing a T3 as $T[x] = \sum_n (\prod_i U_i[n_i, x_i]) \cdot C[n]$ with C = the Tucker core in TT form ($C = TT(G)$), the elementwise product $A \otimes B$ splits cleanly into **two truncatable pieces**:

- **Tucker factors (the "physical legs"):** $W_i[n^A_i, n^B_i, x_i] = U^A_i[n^A_i, x_i] \cdot U^B_i[n^B_i, x_i]$ — i.e. the **COPY-merge over x_i lives here**, as a row-wise Khatri-Rao. Tucker rank $n^A_i \cdot n^B_i$, which you **SVD-truncate** per mode to $\tilde{n}_i$ (the leftover $S^V T$ gets absorbed into the central core).
- **Central core-TT (the bonds):** the new Tucker core is the **outer product** $\tilde{C} = C^A \otimes C^B$, i.e. central cores $\tilde{G}_i = G^A_i \otimes G^B_i$, bonds $r^A \cdot r^B$ — to be **TT-truncated** (rounded).

So my key structural observation: ****in a pure TT, the COPY-merge and the bond blow-up happen at the same carriage, which is exactly what TTM's swap+contract sweep handles together.** In a T3 they separate** — the COPY-merge is *mode-local* (the per-mode Khatri-Rao + SVD on W_i , no sweep), while the bond blow-up is in the **central core-TT**, where the combination is a *Kronecker*, not a COPY-elementwise.**

Where I think TTM maps — and the question for you

My reading is that the generalization applies the TTM **swap+contract+truncate sweep to the central core-TT** (to compress the $r^A \cdot r^B$ bonds without ever materializing them), with two changes from the paper:

1. The "contract" step's **COPY is replaced by the per-mode Tucker combination**: fuse G^A_i , G^B_i , and the W_i -truncation projector P_i ($\tilde{n}_i \times n^A \cdot n^B$) into one central core with tucker leg $\tilde{n}_i$.
2. Because d (number of T3 modes) is small (3-5) vs. the paper's $N=28$, the N^2 swap overhead and the intermediate-bond blow-up should be **much milder** — the benefit is purely the cubic-in- r scaling when the TT bonds are large, plus the ϵ truncation knob.

The one thing I want to confirm before we plan: **do you see the central combination as a Kronecker that TTM compresses (Tucker factors handled separately, mode-local), or do you have a more unified picture** where the COPY also enters the core-TT sweep? That's the hinge — it decides whether "T3-TTM" is "Khatri-Rao the Tucker factors, then run TTM on the core chain" or something tighter.

Tell me your generalization and where it differs from the above, and we'll converge — then look at the existing `t3_mult` and make an implementation plan (named method, tolerance-driven, presumably non-stacking to start, matching the CLAUDE.md note).

Nick

I think this is what I have in mind, but I think about this better in graphical tensor diagram notation than in words. Can you understand a hand-drawn picture? I have posted a hand-drawn diagram of what I am thinking in the project root, the file is T3M_handdrawn_diagrams.jpeg

Claude

Yes, I can read images. Let me look at it.

▸ **Bash** — Confirm the diagram file exists

▸ **Read** T3M_handdrawn_diagrams.jpeg

I can read it well. Let me describe what I see so you can check my parse, then tell you what algorithm I extract from it.

What I see (a $d=2$ worked example)

It reads as the TTM picture, generalized — a 6–7 step reduction of the **two-T3 product network** down to a single compressed T3:

- **Panel 1 (start):** a horizontal chain of **4 central cores** (= the two T3s' core-TTs *concatenated*, A's two cores + B's two reversed — exactly the paper's eq-8 concatenation), with **Tucker-factor circles hanging below** each core, and the dangling physical legs of A's and B's factors meeting at a **triangle (COPY)** at the bottom — i.e. the elementwise join over x_i . A **dashed box** marks the first swap/contract region.
- **Panels 2-3 / bottom row:** the recurring motif is **oval = two tensors contracted/merged (about to be SVD-split), triangle = the COPY merging two Tucker factors over a physical x_i , dashed box = the active swap region** sliding along. Ovals carry a little internal mark in places (the SVD/truncation point).
- **Panel 7 (end):** a clean **2-core chain with 2 vertical Tucker legs** — the output T3.

So: concatenate the **central** TTs, sweep with swap + contract + SVD-truncate, and **fold the Tucker-factor COPY-merge into each contract step**, ending in one compressed T3.

The key thing the diagram settles

The picture answers my earlier open question: it's the **tighter / unified** version, not a separate Tucker pre-pass. The COPY triangles (the $U^A_i \otimes U^B_i$ merge over x_i) live *inside* the swap-and-contract sweep on the core chain, not in a pre-step. The Tucker factors ride along as decorations on the cores' n_i legs and get consumed at the contract where the two same-mode cores meet.

Let me confirm the mechanics (this is where implementation lives)

A few specifics I want to pin down against your drawing:

1. **Swap step** (on the core chain): same as TTM — contract the adjacent pair along the bond, permute, SVD, **truncate to ϵ_{TT}** , move the gauge. The Tucker factor hanging on each core just travels with its n_i leg through the permutation. Right?
2. **Contract step** (when A's and B's same-mode cores meet): three things fuse — (i) the shared central bond, (ii) the two Tucker factors via the COPY over $x_i \rightarrow W_i = U^A_i \otimes U^B_i$, and (iii) an **SVD of W_i truncated to ϵ_{Tucker}** , giving the output Tucker factor $\tilde{U}_i$ plus a small projector $\tilde{n}_i \times n^A \ n^B$ that's absorbed into the output central core. So there are **two distinct truncations** — one on the TT bonds, one on the Tucker rank. Is that how you see it (two tolerances, or one shared ϵ)?
3. **Gauge:** mixed-canonical centered at the rightmost site of the active pair, QR (not SVD) to shuffle the gauge after a contract — carried over verbatim to the core chain?
4. **The COPY's "width":** because the merge is $U^A_i[n^A, x_i] \cdot U^B_i[n^B, x_i]$, the COPY is over the full mode N_i , but it's immediately compressed by the W_i SVD to $\tilde{n}_i \leq n^A \ n^B$. So the wide N_i never propagates into the bonds — it stays local to the contract. Agree?

If those four match your drawing, I think we have the algorithm fully pinned. Next step would be to look at the existing `t3_mult` (the conventional method) to see what infra we can reuse (the COPY/Khatri-Rao, the T3-SVD truncation, gauge sweeps), and then write up an implementation plan — named method, two tolerances, non-stacking to start, per the CLAUDE.md note. Want me to pull up `t3_mult` now, or correct my reading first?

Nick

I think we are in agreement about the algorithm. The principles being: (a) when we swap cores, the tucker factors just follow along for free, and (b) when we fuse same-mode cores, we truncate tucker ranks. However, I just thought about this briefly, and may have made some mistakes or suboptimal choices. Can you please reason about what the “right” algorithm is from a mathematical and computational complexity point of view? During this reasoning, please keep in mind that the number of cores is NOT limited to $d=3-5$ like in my `t4s` paper. This library is intended for a general purpose audience, my `t4s` paper is provided only for your reference and not as a design goal. We want to support the use case where d is large.

Claude

Good — the two principles you state are exactly right. But when I work through the complexity as a *function of d* , the **swap mechanism** for achieving them is the suboptimal part. Let me reason it out.

The complexity of the swap approach is $O(d^2)$ — and that’s avoidable

In the swap layout you concatenate A’s d cores with B’s d reversed cores, then bring same-mode cores together. Same-mode core i starts $\sim d$ positions away, so it takes $i-1$ swaps, total $\sum (i-1) = d(d-1)/2 = O(d^2)$ swaps, each an $O(r^3)$ SVD. So the central-chain cost is

swap-TTM: $O(d^2 \cdot r^3)$ (r = TT bond; quadratic in the number of modes).

Here’s the key observation: ****in an elementwise product, same-mode cores are *already co-located* — both belong to mode i .**** The concatenation deliberately separates them, and the $O(d^2)$ swaps exist only to undo that separation. The swaps are pure overhead from the layout choice. For the paper’s $N=28$ it’s a 378-swap constant; for a general-purpose library with large d it’s the dominant cost and the wrong scaling.

(It’s also where the Appendix-A bond blow-up comes from — long-range permutations turn short-range correlations into long-range ones. No long-range swaps $\implies$ no such blow-up.)

The right algorithm: process in place, left-to-right (a fused multiply-and-truncate)

Never concatenate. Sweep modes $1\dots d$ once, carrying a small remainder R (the already-truncated left bond, size $\tilde{r}$). At mode i :

1. **Tucker merge (your principle b):** $W_i = U^A_i \otimes U^B_i$ (Khatri-Rao over x_i , the COPY), SVD-truncate $\rightarrow \tilde{U}_i$ (output Tucker factor, rank $\tilde{n}_i$) + projector P_i ($\tilde{n}_i \times n^A \times n^B$). Per-mode, $O(d)$ total, independent of the TT bonds.
2. **Central fuse + carry:** contract R with G^A_i , G^B_i , and $P_i \rightarrow$ a tensor $[\tilde{r} \times \tilde{n}_i \times (r^A_{i+1} \times r^B_{i+1})]$; reshape and **SVD-truncate the right bond** to $\tilde{r}'$; emit the output core, carry $R' = S V^T$ forward.
3. Optional **right-to-left cleanup rounding sweep** to make the truncation quasi-optimal (the single forward sweep is greedy).

This is the classic “zip-up + round” used for MPO·MPS / Hadamard products in the TN literature — well-grounded, not novel/risky. Cost:

in-place fused: $O(d \cdot r^4)$ (linear in d).

It never materializes the full product — the Tucker rank $n^A \times n^B$ and the TT bond $r^A \times r^B$ only appear *locally* at the active site and are truncated immediately. Memory $O(\tilde{r} \cdot r^2)$ transiently, never the

global r^2 -bond chain. Both truncations (Tucker $\tilde{n}_i$, TT $\tilde{r}$) are independent SVD knobs (rtol/atol/max-rank), and they can be interleaved for joint optimality (it's morally a fused T3-SVD).

The honest trade-off

There's no free lunch on the bond exponent: the swap's r^3 comes precisely from combining bonds **one at a time** via $\tilde{r} \times \tilde{r}$ SVDs, which the in-place sweep can't do (its site SVD carries the joint $r^A \cdot r^B$ dimension $\rightarrow r^4$). So:

	cost	wins when
swap-TTM	$O(d^2 r^3)$	$r \ll d$ (few modes, huge bonds — the paper's CFD regime)
in-place fused	$O(d r^4)$	$d \gtrsim r$ (and robustly for general / large d)

The crossover is d vs r . **For a general-purpose library that must support large d , the in-place fused sweep is the right default** — $O(d)$ beats $O(d^2)$, and it dodges the swap bond-blow-up entirely. The r^4 vs r^3 only bites in the few-modes/huge-bond corner, which is exactly the niche the paper targeted and *not* the general case.

So my recommendation: **implement the in-place fused multiply-and-truncate** (your two principles, but realized by a left-to-right sweep instead of concatenate+swap). If you later want the $r \ll d$ corner, the swap variant could be added as an option — but I'd start with, and default to, the $O(d)$ one.

One thing I'd check before finalizing: a subtlety in step 2 is whether to truncate the Tucker rank $\tilde{n}_i$ *before* or *jointly with* the TT-bond SVD — doing it jointly (a local T3-SVD at the site) is cleaner and likely what we want. Want me to pull up the existing `t3_mult` now so we can see what infrastructure (Khatri-Rao, `t3svd/truncated_svd`, gauge sweeps) we can reuse, and turn this into a concrete plan?

Nick

Oh, I see. I thought you were confusing my paper (t4s.pdf) with the TTM paper, which is by other people and well-known. But you were not, I understand your comments now. I think the in-place fused approach is essentially what we already have with `__mul__`, except including a rank truncation feature. Can you verify that? If so, I would like to have functionality for both. I.e., we will add rank truncation options to the existing elementwise multiplication code, and we will add the T3 version of the swap-TTM method (let us call this "T3M"). Then we will support both regimes. Two things to keep in mind: 1) beware that `rtol` and `atol` are not compatible with stacking, because the stacked components could end up with different shapes. We need to warn the user in the docs. 2) I prefer to use SVD over QR in all cases for numerical robustness. I understand it is slower and that is OK. I would like to use my existing `truncated_svd` function in `backend/linalg.py`. How does this sound to you? Please assess this plan.

Claude

Let me verify the claim by reading the existing elementwise-multiply code before assessing the plan.

- **Bash** — Locate `t3_mult` and `__mul__`
- **Read** `t3toolbox/backend/t3_linalg.py`

The existing code confirms it — and there's an important nuance to flag. Let me verify, then assess.

Verification: what `t3_mult` actually does

`t3_mult` (`backend/t3_linalg.py:213`) is the **conventional "dumb" method** — its own docstring says so ("does not do intermediate rank truncation... Ideally, we should also implement the newer 'TTM' algorithm"). It **forms the full product**:

- Tucker: $B_{xy} = \text{einsum}('...io,...jo \rightarrow ...ijo', B_x, B_y) \rightarrow \text{rank } n_x \cdot n_y$ (the Khatri-Rao / COPY over the mode).
- Central: $G_{xy} = \text{einsum}('...aib,...ujv \rightarrow ...aujbv', G_x, G_y) \rightarrow \text{bond } r_x \cdot r_y$ (the Kronecker).

So it does the *right combination*, but materializes it at full rank with **no truncation and no fusing**.

This means your statement is right *in spirit* (same combination primitives) but with a crucial caveat about **how** you add truncation — there are two genuinely different things:

- **(a) "form + round"**: call `t3_mult`, then `t3svd-truncate` the result. Trivial (pure reuse), but it *forms the full r^2 -bond, n^2 -rank product* first $\rightarrow O(d \cdot r^6)$ compute, $O(d \cdot r^4 n^2)$ memory.
- **(b) "in-place fused"**: restructure into a left-to-right sweep that truncates each bond *as it forms* (same Khatri-Rao/Kronecker primitives, but with a carried remainder + `truncated_svd` at each site). Never forms the full product $\rightarrow O(d \cdot r^4)$, memory-light.

Your "in-place fused" is (b), not (a) — and (b) is not literally "the existing mult + a truncation call," it's a new truncating sweep that reuses the mult's per-core math. Worth being precise because (a) and (b) differ by two factors of r .

The three methods, by cost (r =TT bond, n =Tucker rank, d =#modes)

method	compute	memory	sweet spot
(a) form+round (mult+t3svd)	$O(d \cdot r^6)$	$O(d \cdot r^4 n^2)$	tiny r ; trivial to ship
(b) in-place fused (zip-up)	$O(d \cdot r^4)$	$O(\tilde{r} \cdot n \cdot r^2)$	large d , moderate-large r
(c) swap-T3M	$O(d^2 \cdot r^3)$	$O(\tilde{r}^2)$	$r \ll d$ (huge bonds, few modes)

(c) is your diagrammed swap method; its $O(d^2)$ is fine *in its own regime* because there d is small. (b) dominates (a) in both compute and memory — (a)'s only virtue is that it's one line.

Assessment of the plan

The two-method split is sound — a truncating elementwise mult for the d -dominated regime + swap-T3M for the r -dominated regime is exactly the right coverage, and the regimes are complementary. The one decision I'd surface: **make the truncating-mult the fused (b), not (a)**. If you want a method that genuinely serves large d , (a)'s $O(r^6)$ /full-product memory undercuts that — (b) is your real workhorse, and (c) handles the $r \ll d$ corner. (a) is worth keeping only as a dead-simple baseline. So I'd frame it as: **(b) in-place fused = default; (c) swap-T3M = option for $r \ll d$** — and optionally leave (a) as the trivial reference path.

Both of your constraints are correct and well-chosen:

1. **rtol/atol $\perp$ stacking** — yes, and it's already the library's convention: `truncated_svd` and `t3svd_dense` *raise* on stacked input with `rtol/atol` (different slices $\rightarrow$ different ranks $\rightarrow$ ragged). `**max_rank truncation is stacking-compatible**` (fixed ranks $\rightarrow$ uniform shapes), so the doc warning should be specifically "rtol/atol require unstacked; max-rank is fine stacked."
2. **SVD over QR via `truncated_svd`** — good, and convenient: `truncated_svd` already does a *full* SVD then slices (I read it earlier), so calling it with no truncation gives you the orthogonalization sweep for free — "SVD everywhere" is literally just `truncated_svd` everywhere. Slower than QR, robust, and zero new code.

So: I'm on board. My one substantive recommendation is **truncating-mult = the fused (b) version** rather than form+round, so the d -regime method is actually efficient. Want me to pull up `t3svd/truncated_svd` and the Khatri-Rao/Kronecker pieces now and turn (b)+(c) into a concrete implementation plan — or do you want to settle (a)-vs-(b) for the first method first?

Nick

Ok, I understand. Let's include all 3: (a), (b), and (c), and provide options to the user to choose one, with (b) being the default. I want to keep (a) primarily for reference, but also because it is parallel rather than sequential. Let's make a concrete implementation plan. We can have backend functions like `t3m_form_then_round`, `t3m_inplace_fused`, `t3m_swap`, then the frontend method is `TuckerTensorTrain.t3m()`, which can use any backend. The `__mul__` method will use (b).

Claude

This is a meaty build. Before I write the plan, let me quickly survey the reusable infrastructure (the implicit `t3svd` signature for (a), and the SVD/orthogonalization primitives I'd reuse for the (b)/(c) sweeps), so the plan is concrete about what's reused vs. new.

▷ **Bash** — Survey reusable SVD/orthogonalization infrastructure

I have what I need. Here's the concrete plan.

API

Frontend (`tucker_tensor_train.py`):

```
def t3m(self, other,                # TuckerTensorTrain, same shape & stack_shape
    method='inplace_fused',        # 'form_then_round' (a) | 'inplace_fused' (b) | 'swap' (c)
    max_tucker_ranks=None,         # len d
    max_tt_ranks=None,             # len d+1
    rtol=None, atol=None,          # require unstacked (raise on stacked)
    ) -> TuckerTensorTrain
```

`__mul__` (the $T3 \times T3$ branch) $\rightarrow$ `self.t3m(other)` i.e. `method='inplace_fused'`, **no tolerances** $\Rightarrow$ **exact** (preserves current $*$ semantics, just via (b)).

Backend (in `t3_linalg.py`, beside `t3_mult`): `t3m_form_then_round`, `t3m_inplace_fused`, `t3m_swap`, each (`x`, `y`, `max_tucker_ranks`, `max_tt_ranks`, `rtol`, `atol`) $\rightarrow$ (`tucker_cores`, `tt_cores`).

Reuse (no new SVD machinery)

- `truncated_svd` (full SVD + slice) — the single SVD primitive everywhere (your SVD-over-QR preference is *free*: it's already full-SVD; calling with no truncation gives the orthogonalization sweep).
- `t3svd` — the rounder for (a).
- The Khatri-Rao / Kronecker einsums already in `t3_mult`.
- `left_svd_pair/right_svd_pair` (`linalg.py`) look purpose-built for SVD-ing a contracted core pair — candidates for the (c) swap.
- Shared validator: `rtol/atol` with stacked input $\Rightarrow$ raise (mirrors `truncated_svd/t3svd`); `max_rank` is stacking-OK.

The three backends

(a) `t3m_form_then_round` — trivial: `t3svd(t3_mult(x, y), max_tt_ranks, max_tucker_ranks, rtol, atol)`. Parallel forming (your reason for keeping it), sequential round. $O(d \cdot r^6)$, memory $O(d \cdot r^4 n^2)$.

(b) `t3m_inplace_fused` — the workhorse, two parts:

- *Tucker (parallel, per mode)*: $W_i = \text{khat_rao}(U^A_i, U^B_i)$; `truncated_svd` $\rightarrow \tilde{U}_i$ (output Tucker factor) + P_i (projector $n^A n^B \rightarrow \tilde{n}_i$).
- *Central (sequential $L \rightarrow R$ sweep)*: carry R keeps r^A , r^B **separate**; at site i contract R with G^A_i, G^B_i , apply $P_i \rightarrow (\tilde{r} \times \tilde{n}_i \times r^A r^B)$, `truncated_svd` the right bond $\rightarrow$ core + new carry. Then a **$R \rightarrow L$ cleanup sweep** for quasi-optimality.
- Never forms the full product: memory $O(\tilde{r} \cdot \tilde{n} \cdot r^2)$, compute $O(d \cdot r^4)$.

(c) `t3m_swap` — the `r⌈d` method: concatenate the two central TTs (A then reversed B , Tucker factors attached to the n_i legs); iterate **swap** (`truncated_svd`, Tucker factors ride along) + **contract** (Tucker COPY-merge W_i + SVD, fuse central) when same-mode cores meet; mixed-canonical gauge via SVD. $O(d^2 \cdot r^3)$, memory $O(\tilde{r}^2)$. Most complex.

Testing (oracle = the dense product)

- **No truncation**: all three `.to_dense() == A.to_dense() * B.to_dense()` to machine precision, and the three agree with each other.

- **With truncation:** $\|result - exact_product\| \leq \text{truncation bound (dropped singular-value mass} \times \text{the } \sqrt{(2d-1)} \text{ t3svd factor)}, \text{ à la the dense-projection truncation test.}$
- **Stacked + max-rank:** per-slice vs the dense oracle. **Stacked + rtol/atol:** raises.
- `test_dispatch`: jit each method with **max-rank** (static shapes); `rtol/atol` stay eager (dynamic).

Phasing (commit per phase, each verified vs the oracle)

1. Scaffold: `t3m()` frontend + the shared truncation-option validator + the dense-oracle test harness + **(a)** backend.
2. **(b)** backend → flip `t3m` default and `__mul__` to it.
3. **(c)** backend.
4. Docs: `t3m` docstring (methods + truncation semantics + "`rtol/atol` require unstacked; `max-rank` OK stacked"); update the `CLAUDE.md` `t3_mult` TODO.

Five decisions I'd like settled before I start

1. **A * B with no tolerances:** have (b) **short-circuit to the parallel form** (= (a) exact) when no truncation is requested, so plain `*` stays fast and exact? (Otherwise (b)'s no-truncation sweep is a slow $O(r^6)$ way to get the same answer.) **Rec: yes, short-circuit.**
2. **Stacking:** support **max-rank stacked**, raise on **rtol/atol stacked** (per your note)? Or defer stacking entirely for v1 per the old `CLAUDE.md` note? **Rec: support max-rank stacked** — it falls out of the '...' einsums.
3. **Tucker vs TT truncation:** keep them **separate** (per-mode Tucker SVD, then the TT sweep) rather than a joint per-site T3-SVD? **Rec: separate** — simpler, and (b)'s cleanup sweep recovers most of the joint optimality.
4. **(b) cleanup sweep:** include the R→L rounding sweep (quasi-optimal) vs. single greedy forward pass? **Rec: include it.**
5. **Priority of (c):** it's the bulk of the work for the narrowest regime — implement now, or land (a)+(b) first and treat (c) as a follow-up? **Rec: (a)+(b) first, (c) as a fast-follow.**

Want me to fold this into a `docs/t3m_plan.md` (like the refactor doc) and start on Phase 1, once you've weighed in on the five decisions?

Nick

1. Yes, short circuit. And this suggests using (a) for `__mul__`, because `__mul__` should work by default on stacked T3s.
 2. Support max-rank stacked.
 3. Having trouble deciding this. Let's talk about it before implementing.
 4. Include cleanup sweep.
 5. Implement all now, stage different commits for each.
- Yes, fold this into `docs/t3m_plan.md`. Before implementing, please provide detailed reasoning for your recommendation for 3. What are the different options for truncation specifications, and what are their pros and cons?

Claude

Good call to settle this before coding — it's the subtlest design point in the build, and it changes the implementation structure. Let me lay it out properly. There are really two coupled questions: **what** the user specifies, and **how** the truncation is performed.

Why this is subtle

A T3 has **two** rank families that both blow up in the product — Tucker ranks $\tilde{n}_i$ (per mode, $n^A n^B \rightarrow \tilde{n}_i$) and TT bonds $\tilde{r}_i$ (per edge, $r^A r^B \rightarrow \tilde{r}_i$). Truncating a rank optimally means keeping the directions with the most **whole-tensor mass**, which for a Tucker leg is " $\tilde{w}_i$'s mass $\times$ the central-TT's weight in that direction." The whole question is whether the Tucker truncation sees that central-TT weight or not.

(1) Specification options — what the user asks for

spec	pros	cons
S1: max_tucker_ranks + max_tt_ranks (fixed)	deterministic shapes $\Rightarrow$ stacking-OK ; predictable cost; no error-budget reasoning	user must <i>know</i> the target ranks (hard to guess for a product); no accuracy guarantee
S2: single rtol/atol, per-step (= t3svd today)	adaptive, accuracy-controlled, reuses t3svd semantics	total error is $\sqrt{(2d-1) \cdot \text{rtol}}$, not rtol; data-dependent ranks $\Rightarrow$ ragged $\Rightarrow$ no stacking
S3: single rtol, global budget (split across the $2d-1$ steps)	total error genuinely $\leq \text{rtol}$	more bookkeeping; still ragged/no-stacking
S4: separate Tucker vs TT tolerances	independent control when the two decay very differently	extra params; combinatorial doc surface

Recommendation on spec: match t3svd exactly — max_tucker_ranks, max_tt_ranks, rtol, atol, per-step (S1+S2). It's the only choice that keeps the three methods comparable to each other and to the existing rounder, and it reuses truncated_svd/t3svd verbatim. S4 can be layered on later if a user needs it; S3 is a nicety we can skip.

(2) Algorithm — separate vs joint (the actual decision)

	Separate (per-mode W_i SVD up-front, then TT sweep)	Joint (per-site, à la t3svd: weighted Tucker SVD + TT SVD interleaved in the sweep)
Tucker truncation quality	truncates $W_i = U^A \otimes U^B$ by its own singular values — unweighted by the central TT $\Rightarrow$ HOSVD-grade (within $\sqrt{d}$ of optimal, but it's truncating the <i>wrong</i> quantity when the TT weighting is skewed)	truncates the Tucker leg weighted by the canonicalized central TT $\Rightarrow$ ST-HOSVD-grade, the right quantity
Consistency with t3svd/(a)	(a) = t3_mult + t3svd is <i>joint</i> ; separate $\Rightarrow$ **(b) compresses worse than (a)** for the same tolerance	matches (a)
Parallelism	Tucker SVDs are independent $\Rightarrow$ vectorizable	Tucker SVDs are sequential (in the sweep)
Memory	~same — both must form W_i ($n^2 \times N$) and pass an n^2 leg through a site once	~same
Complexity	simpler: decoupled passes	reuses t3svd's per-site Tucker+TT logic, but interleaved with on-the-fly product formation

My recommendation: joint — and the clincher is consistency

The decisive argument isn't the $\sqrt{d}$ quality bound in the abstract; it's this: ****(b) exists to be *better* than (a), not worse.**** (a) rounds with t3svd, which truncates the Tucker rank *weighted* (ST-HOSVD). If (b) uses separate/unweighted Tucker truncation, then for the same rtol **(b) keeps larger Tucker ranks than (a)** — the memory-efficient "smart" method would compress worse than the dumb form-then-round. That's backwards and would surface immediately in tests (same tolerance, different ranks, (b) fatter). The only way (b) $\geq$ (a) on compression is to weight the Tucker truncation by the TT — i.e. joint.

And it's cheaper than it sounds: joint **is** the t3svd per-site step (weighted Tucker matricization SVD, then TT-bond SVD), which the library already implements (down_svd_tt_core / left_svd_tt_core etc.). The new work in (b) is only *fusing the product formation into that sweep* — we don't reinvent the truncation. Memory stays light either way (no full r^4 product).

The one honest cost of joint: the Tucker SVDs become sequential (lose the embarrassing parallelism that separate has). But **(b) is already a sequential sweep**, so it gives up nothing it hadn't already given up — and **(a) keeps the fully-parallel form** (your stated reason for keeping it), so the parallel path still exists for users who want it.

So: **(a) joint via t3svd (parallel form + round); (b) joint via fused per-site t3svd-style truncation; (c) joint at each contract/swap.** All three then mean the same thing by rtol, and (b)/(c) dominate (a) on memory without ever being worse on compression.

If you'd rather minimize v1 risk, the fallback is: ship (b) with **separate** truncation first (simpler, gets the TT-bond win which is the memory-critical part), and upgrade the Tucker side to joint as a fast-follow — accepting that until then (b) compresses a bit worse than (a). I'd only do that if joint turns out to complicate the sweep more than I expect.

Does the "(b) must not compress worse than (a)" argument land for you? If so I'll write docs/t3m_plan.md with **joint** locked in and start Phase 1.

Nick

Ok, this makes sense to me. Lock in "joint" for the algorithm. Now for the specification, I believe t3svd currently does not support per-step rtol and atol (and does not support global max ranks). While we are here, maybe we should add support for these options in t3svd.

Claude

Let me verify what t3svd actually supports before we decide what to add — I'll read its implementation (and truncated_svd's capabilities, since t3svd builds on it).

▷ **Read** t3toolbox/backend/t3_svd.py

t3svd *does* pass rtol/atol/max_rank to each per-step SVD (down_svd_tt_core, left_svd_tt_core). So the support might already be there — let me check whether those per-step functions actually *use* them, which is where the real answer is.

▷ **Read** t3toolbox/backend/t3_orthogonalization.py

The chain passes rtol/atol/max_rank down (t3svd → left_svd_tt_core → linalg.left_svd_pair). So the support *looks* present. Rather than trace further, let me settle it empirically — does t3svd actually truncate with max-ranks, rtol, and atol?

▷ **Bash** — Empirically test t3svd truncation options

The scalar (global) max-rank fails (TypeError — only per-mode/edge lists work), confirming that part. But the rtol/atol test was inconclusive: that random tensor is already near minimal rank, so rtol=1e-2 had nothing in the 1%-band to drop. Let me retest rtol/atol decisively with a **loose** tolerance and a **controlled decaying spectrum**:

▷ **Bash** — Decisive rtol/atol truncation test with controlled spectrum

Decisive: t3svd's rtol/atol **do work** — rtol=0.5 → ranks (1,1,1,1)/relerr 0.48, scaling smoothly down to rtol=0.01 → relerr 0.006; atol likewise. So per-step rtol/atol and per-mode/edge max-ranks are all already there. The **only** real gap is the scalar/"global" max-rank (passing one int → TypeError). Let me confirm the stacking behavior too, since it's central to T3M:

▷ **Bash** — Confirm t3svd stacking behavior (max-rank OK, rtol raises)

Complete picture now — and it flips the premise slightly. Here's what t3svd actually supports (all verified empirically):

capability	status
per-step rtol/atol	✓ already works (rtol=0.5→ rank-1, relerr 0.48; tracks the tolerance down to 0.006); atol too
per-mode/per-edge max-rank lists	✓ works
scalar / global max-rank (one int for all)	✗ TypeError: 'int' object is not subscriptable
stacked + max-rank	✓ works (uniform shapes)
stacked + rtol/atol	✗ raises ValueError (the intended constraint)

So the part of your recollection about rtol/atol was off — they're fully functional and per-step. The **one genuine gap is the scalar/global max-rank** (you must currently hand it a full list of length d / d+1). And the stacking rule you flagged is already enforced exactly as we want: max-rank stacks, rtol/atol raise.

Recommendation

Add the scalar-max-rank convenience, via a shared normalizer rather than patching t3svd alone. A tiny `_normalize_max_ranks(spec, length)` (`None`→`None`; `int`→`[int]*length`; `list`→`list`, with a length check) used by `**t3svd` and the three T3M backends`**`, so `max_tucker_ranks=2`, `max_tt_ranks=4` works uniformly everywhere. (For TT bonds a scalar is a *cap* — the boundary `r_0=r_d=1` just truncate to 1 under it, which is correct.) Small, and it keeps the spec identical across `t3svd/t3m`.

So the **T3M truncation spec is settled**: reuse `t3svd`'s exact semantics — `max_tucker_ranks`, `max_tt_ranks` (scalar or list, after the normalizer), `rtol`, `atol`, **per-step**, with `rtol/atol` requiring unstacked. No new tolerance machinery; the joint algorithm runs the per-step SVDs the same way `t3svd` already does.

One thing to confirm so I don't mis-scope: your "per-step" comment makes me want to check — are you happy with **per-step** tolerance semantics (total error $\leq \sqrt{(2d-1) \cdot \text{rtol}}$, what `t3svd` does today), or did you actually want a **global** budget (total $\leq \text{rtol}$, i.e. split the budget across the $2d-1$ steps)? The latter is a separate small feature (the "S3" option from before) we could add to both `t3svd` and T3M if you want it; otherwise I'll leave per-step as-is.

If per-step is fine, then with **joint locked** and the spec being **t3svd's + the scalar normalizer**, I'm ready to write `docs/t3m_plan.md` and start Phase 1 (scaffold + normalizer + method (a)). Shall I proceed — and do you want the global-budget option in scope or not?

Nick

What I mean is the typing of allowed arguments. Currently, for example, `max_tt_ranks: typ.Sequence[int]`, and `rtol: float`. The issue is if we should add support for `max_tt_ranks: int` and `rtol: typ.Sequence[float]`

Claude

Ah, now I understand — it's about making the typings symmetric: `max_ranks` is currently fine-grained-only (a sequence), `rtol` is coarse-only (a scalar), and you're asking whether each should also accept the other form. Here's my read on the two directions, because they're not equally clean:

`max_tucker_ranks / max_tt_ranks: add int → yes`

Clean and useful. "Cap everything at rank k " is a common ask, the broadcast is unambiguous (`int` → `[int]*length`, and for TT bonds the boundary 1s just stay 1 under the cap), and there's no semantic question. Adds nothing but convenience.

`rtol / atol: add Sequence[float] → I'd say no (or, if you really want it, not this way)`

Three reasons it's not symmetric to the rank case:

1. **There's no unambiguous mapping.** `max_tucker_ranks` and `max_tt_ranks` are *already* two separate per-position arrays, so a sequence has an obvious meaning. But `rtol` is ***one scalar feeding both the Tucker SVD and the TT SVD at every step***. A `rtol` sequence has no clear length or target — per-Tucker-mode (d)? per-TT-edge ($d+1$)? per-step in sweep order ($2d-1$, implementation-tied)? Whatever we pick is a convention the user has to memorize.
2. **The genuinely clean symmetric form is a bigger change.** To mirror the ranks properly you'd split into `rtol_tucker` (len d) and `rtol_tt` (len $d+1$), each `float | Sequence[float]` — i.e. four tolerance args (`rtol_tucker`, `rtol_tt`, `atol_tucker`, `atol_tt`) replacing two. That's a real API expansion (and a signature change to `t3svd`) for...
3. **...a niche need.** Exact per-mode *ranks* are a common, concrete thing to specify; per-mode *tolerances* almost never are — you nearly always want one accuracy target. So `max_ranks` legitimately wants both coarse and fine forms, while `rtol` is well-served by the coarse (scalar) form alone. The asymmetry tracks real usage rather than being an oversight.

So my recommendation: `max*_ranks: int | Sequence[int]` (via the shared normalizer), `rtol/atol: float only`. If you later hit a concrete case for graded tolerances, do it deliberately as `rtol_tucker/rtol_tt` (each scalar-or-sequence) rather than overloading the single `rtol` with an ambiguous sequence — that keeps it parallel to the rank args instead of inventing a one-off convention.

(One caveat worth noting: per-step `rtol` *would* be the natural way to hit a global error budget — different `rtol` at each step summing to a target. But as I said before, if global-budget is the actual goal, a single `rtol` + an internal $\pm\sqrt{(2d-1)}$ split is cleaner than exposing the raw sequence.)

If you agree, the settled spec for both `t3svd` and `t3m` is: `max_tucker_ranks/max_tt_ranks` as **scalar or sequence**, `rtol/atol` as **scalars**, per-step, `rtol/atol_stack`. With **joint** locked, shall I write `docs/t3m_plan.md` (including the small shared rank-normalizer + the scalar-max-rank upgrade to `t3svd` as Phase 0) and start building?

Nick

Ok, I agree with your recommendation, and the spec is settled. Please write the `docs/t3m_plan.md`, including the upgrade to `t3svd`. Wait to start building; I want to read the spec first

Claude

Settled. Let me write the full spec to `docs/t3m_plan.md` for your review (and hold off building).

▷ **Write** `docs/t3m_plan.md`

Written to `docs/t3m_plan.md` — and I'm holding (no building, and I've left it uncommitted so you can edit it directly or have me revise before it lands).

It captures everything we settled:

- **The math** (two rank families: Tucker $\tilde{n}_i$ via Khatri-Rao, TT bonds $\tilde{r}_i$ via Kronecker).
- **The three methods** with the complexity table and the d-vs-r crossover — (a) parallel/reference, (b) `inplace_fused` default for large d, (c) swap for $r \ll d$.
- **The five decisions**: joint truncation (with the "must not compress worse than (a)" rationale), the spec (`max*_ranks: int | Sequence[int]`, `rtol/atol: float`, per-step, `rtol/atol_stack`, and *why* we're not adding `sequence-rtol`), SVD-everywhere, `__mul__` → (a) exact/stack-aware, and the no-truncation short-circuit.
- **API** (`t3m()` + `__mul__` + the three backends + the shared `normalize_max_ranks`).
- **Phase 0** = the `t3svd` scalar-max-rank upgrade (noting `rtol/atol` already work, verified), then Phases 1-4 (`scaffold`+(a) → (b) → (c) → `docs`), **separate commits each**.
- **Testing** against the dense oracle, including the joint-quality guard, stacking, and dispatch.
- **Deferred** items (graded tolerances, global budget, uniform/weighted layers, auto method-selection).

Take a look whenever you're ready. Tell me anything you'd like changed — and once you're happy with the spec, I'll commit the doc and start Phase 0.

Nick

Excellent! I have read through the plan and agree completely. Please commit the doc, push it, and begin building

Claude

Approved — committing the plan, then starting Phase 0.

▷ **Bash** — Commit and push the T3M plan doc

Now Phase 0 — the t3svd scalar-max-rank upgrade. Let me read the targets (ranks.py header, dense_t3svd's max-rank usage):

▷ **Bash** — Inspect Phase 0 targets

Now I have the Phase 0 picture. Let me read dense_t3svd.t3svd_dense's rank handling to thread the normalizer correctly there:

▷ **Read** t3toolbox/backend/dense_t3svd.py

O input handling is clear. Let me implement Phase 0. First, the shared normalize_max_ranks helper in ranks.py:

▷ **Read** t3toolbox/backend/ranks.py

▷ **Edit** t3toolbox/backend/ranks.py

Now thread it through backend t3svd — add the import, normalize at the top, simplify the per-step lines:

▷ **Edit** t3toolbox/backend/t3_svd.py

▷ **Edit** t3toolbox/backend/t3_svd.py

▷ **Edit** t3toolbox/backend/t3_svd.py

▷ **Edit** t3toolbox/backend/t3_svd.py

Now dense_t3svd — check its imports and thread the normalizer:

▷ **Bash** — Check dense_t3svd imports

▷ **Edit** t3toolbox/backend/dense_t3svd.py

▷ **Edit** t3toolbox/backend/dense_t3svd.py

Core scalar support is in (helper + both backends). Let me verify it works (scalar $\equiv$ broadcast list, for both t3svd and t3svd_dense, via the frontend) before polishing the frontend hints:

▷ **Bash** — Verify scalar max-rank works through t3svd and t3svd_dense

Test bug — the frontend t3svd returns a TuckerTensorTrain directly (not data). Fixing:

▷ **Bash** — Re-verify scalar max-rank support

Scalar $\equiv$ list confirmed for both. Now let me update the frontend type hints/docstrings to advertise the scalar option. Let me read the t3svd and t3svd_dense signatures + param docs:

▷ **Read** t3toolbox/tucker_tensor_train.py

▷ **Edit** t3toolbox/tucker_tensor_train.py

▷ **Bash** — Check Union import and t3svd_dense signature

Union isn't imported (only typ.Union is available via typing as typ). Let me fix my edit:

▷ **Edit** t3toolbox/tucker_tensor_train.py

Now the docstring params for t3svd, and the t3svd_dense signature + docstring. Let me locate and read them:

▷ **Bash** — Locate t3svd/t3svd_dense signatures and docstring params

▷ **Edit** t3toolbox/tucker_tensor_train.py

Both t3svd and t3svd_dense share this docstring — updating both:

▷ **Edit** t3toolbox/tucker_tensor_train.py

▷ **Edit** t3toolbox/tucker_tensor_train.py

Now the t3svd_dense signature hints (line 3528-3529):

▷ **Edit** t3toolbox/tucker_tensor_train.py

Now let me verify no regression — run the full suite (t3svd is used widely: retract, etc.):

▷ **Bash** — Run full suite after Phase 0 changes

All 150 pass — no regression. Now let me add a test for the scalar max-rank support. Let me find the existing t3svd test:

▷ **Bash** — Find t3svd tests

▷ **Read** tests/test_tucker_tensor_train.py

Let me check the test helper (check_relerr) exists, then add the scalar-max-rank test:

▷ **Bash** — Check for check_relerr helper and ranks import

▷ **Edit** tests/test_tucker_tensor_train.py

▷ **Bash** — Run the new scalar-max-rank test

Phase 0 test passes. Let me commit Phase 0 (the t3svd scalar-max-rank upgrade).

▷ **Bash** — Commit and push Phase 0

Phase 0 committed (f157adac). The stray TTM.pdf/diagram/.npz are correctly untracked (not added). Now **Phase 1** — scaffold + method (a). Let me set up the backend. First, t3_linalg.py's header (to add the import + __all__):

▷ **Bash** — Read t3_linalg header and check for import cycle

No cycle. Implementing Phase 1 backend — t3m_form_then_round in t3_linalg.py:

▷ **Edit** t3toolbox/backend/t3_linalg.py

▷ **Edit** t3toolbox/backend/t3_linalg.py

Let me verify the import chain is clean before adding the frontend:

▷ **Bash** — Verify no import cycle from t3_linalg -> t3_svd

Clean. Now the frontend t3m() method. Let me read __mul__ to add t3m() nearby and route through it:

▷ **Read** t3toolbox/tucker_tensor_train.py

Now route __mul__ through t3m and add the t3m() method:

▷ **Edit** t3toolbox/tucker_tensor_train.py

Now the test harness — tests/test_t3m.py (reusable across phases) + method (a) tests:

▷ **Write** tests/test_t3m.py

▷ **Bash** — Run the new t3m tests

Phase 1 tests pass. Let me run the full suite (the __mul__ re-routing touches a widely-used path):

▷ **Bash** — Run full suite after Phase 1

All 154 green. Let me confirm no new doctest failures in tucker_tensor_train.py (I touched __mul__ + t3svd docstrings):

▷ **Bash** — A/B doctest check on tucker_tensor_train.py

No doctest regression (190 = 190). Phase 1 complete. Committing:

▷ **Bash** — Commit and push Phase 1

Phase 1 committed (e64705d7). Now **Phase 2** — **(b) inplace_fused**, the algorithmically substantial one. My approach: right-orthogonalize the two central TTs separately (the Kronecker of right-canonical cores is right-canonical, so the product's central TT is implicitly right-canonical — no need to form it), then a single optimal left-to-right fused sweep with a separate-bond carry. Let me read the right-orthogonalization helper and re-confirm truncated_svd's interface:

▷ **Bash** — Read right_orthogonalize_tt_cores

right_orthogonalize_tt_cores(tt_cores) is what I need. Now let me implement the fused sweep. First the imports, then the function. Let me add the imports to t3_linalg.py:

▷ **Edit** t3toolbox/backend/t3_linalg.py

Now the fused-sweep function:

▷ **Edit** t3toolbox/backend/t3_linalg.py

Now wire `inplace_fused` into the frontend (and flip the default per Phase 2):

▷ **Edit** t3toolbox/tucker_tensor_train.py

The git stash earlier reset the read state. Let me re-read and edit:

▷ **Bash** — Locate t3m default and dispatcher lines

▷ **Read** t3toolbox/tucker_tensor_train.py

▷ **Edit** t3toolbox/tucker_tensor_train.py

▷ **Read** t3toolbox/tucker_tensor_train.py

▷ **Edit** t3toolbox/tucker_tensor_train.py

Now add the sweep-exact harness check and the `inplace_fused` test, then run:

▷ **Edit** tests/test_t3m.py

▷ **Bash** — Run the `inplace_fused` test

It passes first try — the fused sweep is correct (short-circuit, generous-rank sweep path, and truncation all check out). Let me run the full `test_t3m` and broader suite for regressions:

▷ **Bash** — Run t3m + broader suite

All 103 pass. Phase 2 (b) is correct — and notably, `check_truncated` passing within $2\times$ the *optimal* dense reference confirms the truncation is joint-quality (a separate/HOSVD Tucker truncation would overshoot). Committing Phase 2:

▷ **Bash** — Commit and push Phase 2

Phase 2 committed (f6deae43). Let me checkpoint here before diving into the swap method, because it's the intricate one.

T3M progress — Phases 0-2 done and pushed

commit	phase	what
f157adac	0	t3svd scalar max-rank upgrade (shared normalize_max_ranks; rtol/atol already worked)
e64705d7	1	t3m() scaffold + method (a) <code>form_then_round</code> ; * routes through it; dense-oracle test harness
f6deae43	2	method (b) <code>inplace_fused</code> — the default; the fused right-canonical sweep, never forms the full product

Methods (a) and (b) are implemented, tested against the dense oracle (exact no-truncation, generous-rank sweep path, and truncation within $2\times$ the optimal reference $\Rightarrow$ confirmed **joint**-quality per Decision 1), and the full suite is green. The default t3m method is now `inplace_fused`; * stays pinned to (a) (exact, stack-aware). The dispatcher cleanly raises `NotImplementedError` for 'swap', so the tree is in a clean working state.

Remaining: Phase 3 (swap) + Phase 4 (docs)

Phase 3 — (c) swap is the genuinely complex one: it's the TTM swap algorithm (concatenate the two central TTs, then $O(d^2)$ swaps to bring same-mode cores together + contract), and a *correct, memory-efficient* version needs **mixed-canonical gauge management during the truncating swaps** (otherwise the swaps either lose accuracy or blow the intermediate bonds — and a no-truncation version would defeat the whole point of (c)). That's a meaty, iteration-prone piece, more involved than (b).

(b) — the default and the right method for general/large d — is already done, so nothing common is blocked by (c); it's purely the $r \ll d$ specialist.

I deliberately paused here rather than rush (c) at the tail of a long session. Two options, your call:

- 1. **Continue now** — I'll implement (c) (gauge-managed truncating swaps) + Phase 4 docs.
- 2. **Bank this checkpoint** and do (c)+docs as a focused follow-up.

Either way the codebase is clean and (a)/(b)/the t3svd upgrade are live. Which do you prefer?

Nick

Let's bank this checkpoint now. We are running out of context. Please update the relevant documents such as the t3m plan and CLAUDE.md, and write a handoff document

Claude

Banking the checkpoint. Let me update the three documents. First, the status in docs/t3m_plan.md:

▷ **Edit** docs/t3m_plan.md

Now the CLAUDE.md TODO. Let me locate the t3_mult line:

▷ **Bash** — Find t3_mult TODO in CLAUDE.md

▷ **Read** CLAUDE.md

▷ **Edit** CLAUDE.md

Now the handoff document:

▷ **Write** docs/t3m_handoff.md

Let me confirm only the docs are pending, then commit:

▷ **Bash** — Confirm pending changes are docs-only

All code is committed; only the docs are pending (the conf_OLD.py/make.bat are pre-existing strays). Committing the checkpoint docs:

▷ **Bash** — Commit and push the checkpoint docs

Checkpoint banked and pushed. Everything is in a clean, working state.

What's committed (5 T3M commits)

commit	what
c05689f1	T3M plan doc
f157adac	Phase 0 — t3svd scalar max-rank (normalize_max_ranks)
e64705d7	Phase 1 — t3m() scaffold + method (a) + test harness
f6deae43	Phase 2 — method (b) inplace_fused (the default)
72d19973	Checkpoint — docs updated + handoff written

Docs updated

- docs/t3m_plan.md — per-phase status (0-2 ✓ with commit hashes, 3-4 ☐) + a Status line.
- CLAUDE.md — the cleanup-backlog t3_mult line replaced with an in-progress **T3M** entry (what's done, what remains, the spec, pointers to the plan + handoff).
- docs/t3m_handoff.md (new) — a concrete resume guide: state/commits, where the code lives, the settled decisions (don't relitigate), the **Phase 3 (c) t3m_swap** algorithm with the gauge-management caveat called out as the hard part + what to reuse from (b), the **Phase 4** doc/test polish (cross-method joint test, test_dispatch jit cases), and the reminder to leave TTM.pdf/the diagram untracked.

State for whoever resumes

- Methods **(a)** and **(b)** work and are tested; **(b)** is the default, * pins (a). Full suite green.

- 'swap' cleanly raises `NotImplementedError` — no broken/half-built code.
- Picking up = implement `t3m_swap` (the gauge-managed truncating swaps; reuse (b)'s per-site joint-truncation block), wire it into `t3m`'s dispatcher + `__all__`, and add a `test_swap` mirroring `test_inplace_fused`. Then Phase 4 polish.

Good place to stop. Thanks — this was a genuinely fun stretch of work.

Nick

Excellent! Thank you for all the work today, this was great.

Claude

You're welcome — I enjoyed it. It was a genuinely great session across a lot of ground:

- **Backend separation refactor** — finished the interrupted work, then applied the backend/frontend razor to move 5 more methods cleanly.
- **Dispatch cleanup** — fixed `project_dense_onto_tangent` and the `raw-np.*` residual functions.
- **Contraction-only dense tangent projection** — reasoned it out, proved minimal-rank isn't required, verified numerically, and shipped it with both methods.
- **T3M** — worked through the TTM paper and your diagram together, settled the algorithm/complexity/spec decisions, and landed Phases 0–2 (the `t3svd` scalar upgrade + methods (a) and (b)) with a clean handoff for the swap method.

Everything's committed and pushed, the suite is green, and `docs/t3m_handoff.md` + the updated `CLAUDE.md/plan` will make resuming the swap method straightforward.

Thanks for the careful collaboration — the reason-first-then-verify rhythm made it a pleasure. Enjoy the rest of your day, and good luck with Phase 3 whenever you pick it back up.